

## RESEARCH ARTICLE

# Building a Comprehensive Trust Evaluation Model to Secure Cloud Services From Reputation Attacks

**SALAH T. ALSHAMMARI<sup>1</sup>, MUNA AL-RAZGAN<sup>2</sup>, TAHA ALFAKIH<sup>1</sup>,  
AND KHALID A. ALGHAMDI<sup>1</sup>**

<sup>1</sup>Department of Information Systems, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia

<sup>2</sup>Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh 11345, Saudi Arabia

Corresponding author: Salah T. Alshammari (salahtshammari@gmail.com)

This work was supported by the King Saud University, Riyadh, Saudi Arabia, under Project RSP2024R206.

**ABSTRACT** Data owners can store extensive data in a flexible and cost-efficient manner by utilizing cloud data storage. Thus, the number of online cloud service providers and their users have undergone a substantial increase. A significant concern in the field of cloud computing is the security and confidentiality of data. A cryptographic access control technique can be used to guarantee data privacy, which prevents unauthorized access and allows only authorized users to access the data. Nevertheless, the trust issue is not fully addressed in this kind of cryptographic approach. The study primarily aims to identify a solution that effectively addresses the trust issues within access control methods, thereby reducing the security risks associated with cloud storage systems. This, in turn, enhances the quality of decisions made by data owners as well as cloud operators. In this paper, we proposed integrating task-role-based access control (T-RBAC) as a new cloud access control model with a comprehensive trust model using various criteria to offer high security for cloud storage systems, with taking flexibility into consideration. There are trust-based systems integrated with various existing access control models in cloud computing; however, these models do not provide accurate trust value computations, and may still be at risk of certain security threats. Hence, these threats can be eliminated through an accurate trust-based T-RBAC model.

**INDEX TERMS** Cloud computing, cloud computing security, cyberattack, data security, information security, secure storage, security, security management, trusted computing, trust management.

## I. INTRODUCTION

This paper proposes a novel integration of the T-RBAC model with a dynamic trust evaluation mechanism to enhance cloud security. We aim to address gaps in existing models by providing a robust solution that dynamically adjusts access based on trust assessments. This effort is driven by the recognition that when assessing the reliability of tasks and roles, trust models encompass both hierarchy and lineage. In addition, they should serve as the foundation for the envisioned design of a trust-based cloud storage system. Using this design, a trust model can be integrated into a cloud computing storage system in which a cryptographic T-RBAC approach is used.

The associate editor coordinating the review of this manuscript and approving it for publication was Sedat Akleyek<sup>1</sup>.

Cloud computing security remains a critical concern across various industries, including healthcare, finance, and government sectors. Our proposed T-RBAC model integrated with a trust evaluation mechanism offers enhanced security and privacy controls essential for these applications. For instance, in healthcare, our model ensures that sensitive patient data is accessible only to trusted personnel with the appropriate roles, dynamically adjusting access based on real-time trust assessments.

Digital data owners are empowered by the incorporation of cryptographic T-RBAC into the trust model within cloud computing storage systems to control data access and decoding permissions. Thus, the consumers are responsible for demonstrating high levels of trustworthiness and persuading service providers to grant them access to data. In the proposed

framework, it is very important to consider hierarchy and legacy when assessing consumer trustworthiness. This integration of the trust model with T-RBAC yields three potential models for improving cloud computing storage systems: Recommender Owner-Role, Consumer-Role, and Consumer-Task-based T-RBACs. In these models, data owners can assess the trustworthiness of roles and tasks, which may assist in assessing the trustworthiness of a user. However, it is worth noting that these three models face certain security concerns, such as being vulnerable to reputation attacks [1], [2]. Many authors have put forth different techniques for improving the security of cloud data storage systems, so that their efficiency and quality is increased. Nevertheless, there are a few security risks that continue to remain a challenge [3], [4], [5].

In general, cloud services are provided in public environments; thus, it is very important to ensure the security of these services [6] [7]. Public platforms can be accessed by many users, both within or beyond a network, which is why it is essential to secure these services. Sometimes, those who have stored their data in the cloud seek high levels of privacy, not only from the public, but also from those providing cloud services. In such situations, security is provisioned using access control methods [8], [9], [10]. System designs and access control policies have spurred several studies in attempts to find the most suitable cloud computing frameworks. This study has introduced the integration of T-RBAC with an exhaustive trust model as a new, flexible access control method through the application of different criteria geared towards the provision of a high degree of security for cloud storage systems. The mode of operation of T-RBAC ensures the assignment of single or multiple roles to each and every user in the system, each role in which is composed of a variety of tasks in a single enterprise set. The use of T-RBAC is, however, not convenient in a distributed environment, even though it offers a security mechanism that provides physical server support for the structure of the security network. Improving security within a cloud computing environment must necessarily employ trust models in order to heighten the security and privacy of data available in a cloud storage system.

When selecting an appropriate access control model, several aspects need to be considered, which include affordability, effectiveness in averting misconduct, security, cost-effectiveness, and trust created following the use of the model. A very important part is played by trust in ensuring the security of cloud computing services [11], [12], and according to [13], "it is an inherent feature that exists in the background, rather than being explicit, and hence well-defined and measurable". Role-based access control (RBAC), discretion access control (DAC), attribute-based access control (ABAC), and mandatory access control (MAC) are some of these models [14], [15], [16]. However, a more convenient approach is integrating trust in cryptographic T-RBAC to improve data security in the cloud. This also creates greater interactions between the cloud consumers' roles and responsibilities in terms of the owners' data. It is

also essential to have a control technique that creates trust in terms of the tasks that individual users carry out.

The T-RBAC approach is presented in this study as a new access control model that incorporates a comprehensive trust model using various criteria to offer high security for cloud storage systems, with flexibility also taken into account [17], [18]. The functioning of T-RBAC is such that all system users are assigned a single role or multiple roles, with every role having various tasks as part of an enterprise setting. Although a sophisticated security mechanism is offered by T-RBAC that offers security to a network through physical servers, it is not an appropriate method for distributed settings. Security in a cloud environment can be enhanced by applying trust models that increase the security and confidentiality of data stored within a cloud storage system [19], [20].

**Discretionary Access Control:** This involves using a subject's identity, like a username or group membership, to control access to particular data. DAC allows the data owner to pass the right of access to another subject at will, making the model more flexible in commercial solutions [21]. However, in such a mode of operation, it is impossible for the management to control the flow of data in the systems, making it unsuitable for environments that require rigorous data flow control with absolutely no data leakage.

**Mandatory Access Control:** This is used by the central authority or the manager of the system to control access to stored data, but not by the owner of the data. This reduces the probability of potential illegitimate data access in comparison with the DAC model, as the flow of data is meticulously controlled [21]. Nevertheless, MAC's level of rigidity is not suitable for commercial domains. Moreover, the model is vulnerable to collusion attacks whereby a data sender is made to collude with the data receiver.

**Role-Based Access Control:** This involves the user's organizational role to control access to data. Access rights are assigned to roles which are then mapped to the user rather than assigning the access rights directly to users [21]. As such, malicious operations or unauthorized access requests can be subjected to some constraints. This enables RBAC to prevent data leakages to unauthorized entities, thus reducing the fraud risks. It also enhances integrity, as permission is imperative to any form of data modification and is hence suitable for highly distributed ecosystems [5], [22]. However, the RBAC model employs a rather static access control policy regardless of a change in the circumstances of the business environment. Secondly, the model does not clearly spell out the difference between the concepts of 'roles' and 'tasks'. Finally and most importantly, the RBAC model fails to take the issue of trust into consideration, an aspect that has provoked further research on models that better value trust. The permissions in the RBAC model are awarded to the role, and the permission for these roles is allocated to particular users [23], [24], [25].

**Task-Role Based Access Control:** The T-RBAC approach enables authorized consumers to access data but denies

unauthorized users the opportunity to reach the data. Nonetheless, such a cryptographic approach is not capable of addressing issues pertaining to trust. This model was chosen for this research due to the fact that the concept of trust is highly subjective and is capable of causing serious problems, as users' thinking and opinions have a significant influence on the precision of an interaction trust evaluation. Finally, the T-RBAC approach is more convenient to use and for developing trust among cloud users of the roles and tasks involved and the owners of the data. There is also a need for a control method that will develop trust regarding the tasks carried out by individual users. The permissions in the T-RBAC model are assigned to either roles or tasks [26], [27]. If tasks are allocated permissions, then roles are assigned to the tasks; hence, a user can acquire permission to access cloud storage through either tasks or roles. Roles in our trust model signify the users' roles for all users with permission to access the service data, while a task signifies a single operation of the service data, and is the smallest unit in our architecture.

#### A. RESEARCH MOTIVATIONS AND CHALLENGES

This research was motivated by the need to develop a trust model that considers hierarchy and inheritance when it comes to the evaluation of the trustworthiness of roles and tasks taking place in cloud data storage systems. The need to provide permanent solutions for a variety of attacks. The dynamic nature of cloud environments necessitates an adaptive trust evaluation mechanism to maintain robust security. Current models inadequately address the rapid changes in trust levels, leading to potential security breaches. Our model leverages real-time trust assessments to dynamically adjust access controls, providing a more resilient and adaptive solution. This approach is crucial for environments where trust and role assignments are fluid and need constant reassessment. Above all, there was the need for a model that is capable of applying various criteria such as interaction importance, conditional transfer, tasks trust, trust decline, and subjectivity in order to provide better security for cloud data storage systems. In our system, we take the feedback as a percentage to remove the problem of non-precise results. Among the challenges is the need for big data in order to test all the criteria and ensure they are correct. Furthermore, we need to ensure that no threats exist. Another challenge is related to constant monitoring of both the compliance and operational effectiveness of the access control model to identify the possibilities of potential security threats, which is not only complex but also time-consuming.

A trust-based T-RBAC model for the cloud environment is presented in Figure 1. There are four key parties in a T-RBAC model. The first are the data owners, who usually share or store their information or resources in the cloud, with the second being consumers who need access to data owners' shared resources. The roles that offer access levels to registered cloud users represent the third party, with the tasks assigned to each role being the fourth. Three potential models

emerge from integrating the trust model with the T-RBAC model in improving cloud computing storage systems, which are the recommender owner-role, the consumer-role, and the consumer-task-based T-RBAC. The trust value of the roles and tasks can be determined by the data owners, which leads to the assessment of users' trustworthiness in the three models.

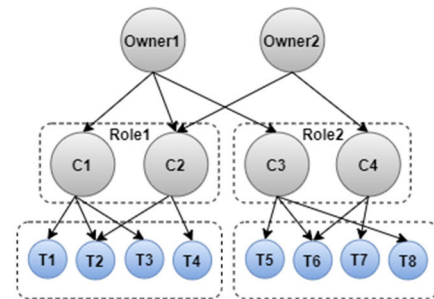


FIGURE 1. Trust-based T-RBAC model.

The T-RBAC system model works as follows. The owner, i.e., the service provider (SP), first shares the resources/services with the roles based on the reputations and trust levels of the tasks and roles. When the trust level of the registered users in the roles is greater than the trust acceptance threshold of the roles, they can gain access to the shared services/resources. Also, access to shared resources or services is provided to registered users in tasks when their trust level is higher than the trust acceptance threshold of the tasks. There will be changes in the reputations of the consumer roles or tasks with changes in the trust levels of the service provider feedback, and this will have an impact on the users' privileges. In this system, there are five types of trust: owners' trust in the recommendations of other owners; owners' trust in the functionality of tasks; owners' trust in the functionality of roles; roles' trust in terms of users' functionality; and tasks' trust in users' functionality. Accordingly, the T-RBAC approach was chosen for the following reasons:

- It offers the greatest extent of flexibility regarding which the interactions of the cloud environment are based, which improves cloud users' interactions in tasks and roles related to owners' data.
- It establishes trust in terms of the tasks performed by consumers.
- Existing trust-based systems for various access control models do not provide accurate trust-value computations, and may be at risk of security threats. An accurate trust-based T-RBAC model can address these threats.
- Dynamic real-time security management is supported by T-RBAC, and the idea of access control over tasks, which are the smallest units for cloud computing activities, is employed.

#### B. PROBLEM STATEMENT

A significant area of concern is the issue of authorization for accessing cloud computing storage, particularly when

there are several users and highly sensitive big data of cloud computing. Consequently, access controls are frequently implemented on cloud computing server application platforms. However, it has been determined that reliability is lacking in access controls related to distributed systems. This deficiency primarily stems from the dynamic and complicated nature of the user population and the inability to establish their identities beforehand. Control models can be integrated to address these concerns, where a more viable option for decentralized systems is trust models; these models have evolved from numerous efforts by trust model developers to create innovative trust models that can effectively solve the most intricate and sophisticated authorization challenges [3], [28], [29]. Several proposals have been put forward for integrated trust models with access controls, but these models are all relatively vulnerable to some attacks. This paper therefore involves a proposal for a new access control model for cloud data storage systems which is called T-RBAC. However, advancements of security threats that occur over time should be considered.

### C. CONTRIBUTIONS

The focus of this paper is on the T-RBAC model and how it is applied to access control to offer security for cloud data storage. The key contribution is that T-RBAC makes use of cryptography to provide protection for data within a cloud storage system. Various studies using many access control models have been carried out in the past on trust management systems. The T-RBAC in this research is a new access control approach integrated with a comprehensive trust model by employing various criteria to offer flexibility and robust security for cloud storage systems. Using tasks and roles in combination with cryptography offers greater security for data stored in cloud systems.

Consumer trust is dependent on previously completed tasks and roles, and the purpose of this research is to study the fundamental effects of task hierarchy and inheritance on trust in consumer roles and tasks. We integrate criteria that need to be applied in the use of a trust-based system and recommend a comprehensive and acceptable T-RBAC model to provide privacy and security for data in a cloud storage system. The advantages of a T-RBAC model are that it is suitable for computing activities that are distributed, such as those having multiple access points with controls, and that it lays the fundamentals for active security models.

The T-RBAC model's application extends to various current scenarios. In financial services, it provides robust protection against internal and external threats by ensuring that only trusted individuals with the appropriate roles can access critical financial data. Similarly, in government systems, the model secures sensitive information and supports compliance with stringent regulatory requirements by continuously evaluating trust and dynamically adjusting access controls. This can be done effectively by using a trust model that is able to do the following:

- Protect data in the cloud from various types of attacks.

- Uphold the privacy of cloud consumers' information because it may contain highly sensitive data.
- Apply the interaction importance criterion to make the value of consumers' trust precise.
- Guarantee that the trust management service is available due to the dynamic nature of cloud services.
- Apply multiple criteria to ensure the accurate assessment and protection of data.

## II. RELATED WORKS

This section presents an overview of several research prototypes in the field of trust management for cloud computing. Ghafoorian et al. [1] put forward the distinct trust and reputation-based RBAC model, which is not only capable of effectively managing the security risks inherent in trust-based RBAC models but is also scalable due to its efficient execution time. Zhou et al. [30] put forth a trust-enhanced cryptographic role-based access control model aimed at enhancing the security of data stored in cloud systems utilizing cryptographic RBACs. Reputation- and recommendation-based trust management for cloud services is supported by this model, with a specific focus on the perspectives of service providers. Noor et al. [31] introduced the CloudArmor model, which streamlines reputation-based trust management for cloud services. This model takes into account the perspectives of both cloud service customers and providers. In addition, it supports cloud service trust management that is based on both reputation as well as prediction. Alhamad et al. [32] used SLA information to develop a trust model for cloud computing that allowed policy-based trust management of cloud services. This model stresses the perspectives of cloud service customers. In addition, it describes the requirements and benefits of utilizing SLA for trust modeling in cloud environments. Santos et al. [33] presented the Trusted Cloud-Computing Platform (TCCP), which supports policy-based trust management of cloud services and provides a secure execution environment for IaaS services. Additionally, TCCP ensures the confidential utilization of guest virtual machines. Our previous works, cited as [17] and [18], laid the groundwork for integrating trust evaluations with access control. This paper extends those concepts by introducing a dynamic and real-time trust assessment mechanism, significantly enhancing the adaptability and security of the T-RBAC model. Hwang et al. [34] and Hwang and Li [35] proposed a security-aware cloud architecture that utilizes VPN and SSL for secure communication. The architecture addresses trust management from both cloud service providers' and customers' perspectives, enabling policy-based trust management. It employs fuzzy logic and Public-Key Infrastructure (PKI) for provider verification and uses Distributed-Hash Table (DHT)-based trust-overlay networks for customer reputation management. Although labeled reputation-based, it primarily relies on policy-based verification through feedback from trusted participants. Uikey and Bhilare [36] introduced TrustRBAC, a role-based access control model for single- and multi-domain cloud



settings, based on trust management. This model supports recommendation- and reputation-based trust management, focusing on customers' perspectives. It uses security policies to determine direct and recommendation trust and features a decentralized configuration where service requesters evaluate provider reputations. While useful, the model is vulnerable to collusion attacks.

### III. DESIGN AND METHODS

This segment discusses the various components of trust models and the role they play in ensuring that the overall structure performs properly. TMS is part of the proposed specification that assesses the degree to which cloud service providers are able to trust users. The trust protection system is made up of several sub-sections, each with its own set of obligations and all of which are aimed at maintaining the protection and privacy of data stored in cloud storage services. The suggested framework architecture is depicted in Figure 2.

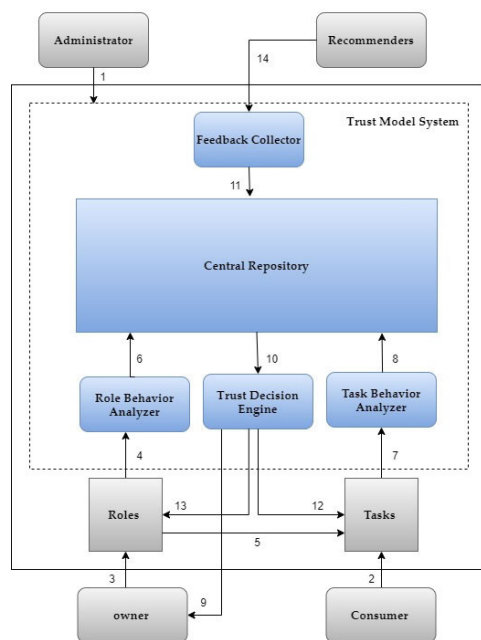


FIGURE 2. Trust model system architecture.

The confidence model consists of all the units in the cryptographic T-RBAC model, as shown in the diagram above. The cloud administrator creates the system's certificates and gives the user all required access credentials. Additionally, the administrator is responsible for the system's organizational arrangement. The administrator creates criteria that reflect a role's location in the system. The hierarchical roles that are created are saved in the cloud environment and are open to all consumers who require them. The roles' entity describes the consumers' memberships and acts as a link between tasks and consumers. Each function has a role parameter that determines the user's membership. Tasks are entities that bind functions and users, and each task has its own criteria for determining the smallest operation based on

user membership. Owners are companies or organizations that own data and choose to keep it encrypted in the cloud so that users can access it easily, and they are in charge of assessing who has access to their data based on the task/role-based policies. A data owner may still be a user, but for an owner-user to view the information contained in a device, they must first request it from the cloud, which then decrypts the data for them. Consumers, also known as service consumers, are secondary entities that require access to shared facilities or infrastructure. The proposed trust model is split into five parts, as described below.

**The Trust Management System (TMS):** This is a layer of the trust system aimed at ensuring that the whole system functions properly. It is made up of various elements that will be addressed later.

**The Central Repository:** This can be thought of as a store of interaction experiences. It is primarily used to store all trust records and interaction histories created when a consumer communicates with roles and tasks, which are then used by the trust decision engine to determine the roles' and tasks' trust values.

**Role Behavior Analyzer:** This entity is in charge of examining interactions and rules relating to minimum trust-level legislation when accessing shared services. It computes a confidence value and stores it in the central repository based on the data owners' feedback. This analyzer is connected to the role entity in order to collect information about roles when a report on a data leak is generated. It easily identifies attackers or unauthorized users, and provides evidence of any data leakage. It also checks to see whether any licensed and recognized consumers' accounts have been involved in such an incident.

**Task Behavior Analyzer:** This entity is in charge of examining activities and interactions relating to minimum trust-level rules when accessing shared services. It computes a calculated confidence score and stores it in the central repository in order to evaluate the tasks defined within the trust level based on data owners' feedback. To define and monitor the tasks undertaken, the task behavior analyzer has to specify authorized users. It, too, easily identifies attackers or unauthorized users and provides evidence of any data leakage. It also checks to see whether any licensed and recognized customers' accounts have been involved in such an incident.

**Feedback Collector:** This agent controls and distributes input from owners to the central repository, since feedback on roles and tasks reflects a consumer's trustworthiness. The information gathered is saved in a central archive for future reference. The feedback receiver preserves the confidentiality of feedback on assignments and functions for security purposes. This component's job is to ensure that only registered users can upload comments into the system.

**Trust Decision Engine:** This entity is responsible for assessing the trust levels of data owners, roles, and individual tasks. It gathers most data regarding interaction records from the centralized database and then decides how the system should respond to access requests.

The administrator initially activates the system and determines its hierarchy of roles and responsibilities. The process of uploading the system's parameters for roles and tasks to the cloud is simplified by Channel 1.

- Depending on their tasks and responsibilities, consumers seeking access to cloud data use Channel 2 to initiate an access request.
- In case a consumer's request is granted, the role entity employs Channel 5 to transmit the request to the task entity. The cloud then responds by presenting a cryptographic T-RBAC plan to the consumer.
- The owner's trust in the role or task determines whether authorization for encryption and data uploading via Channel 3 is given or not. In this stage, the owner also communicates the consumers' identities to the feedback collector.
- If an untrustworthy consumer indulges in data leakage, the owner is required to use Channel 14 to report the role or task to the feedback collector.
- When new feedback is provided by an approved owner, Channel 11 is used by the feedback collector to transmit it to the centralized database, where each confidence report and interaction record generated during role and task interactions is archived.
- Consequently, the interaction logs are stored in a centralized database, which the trust decision engine uses later to assess the trust value of roles and tasks through Channel 10.
- The role entity may endorse role trust evaluations from the TMS at any point, and in response, the role entity requests feedback from the trust decision engine through Channel 13.
- The parameters that determine a consumer's cloud role membership are examined by the role entity till feedback from the trust decision engine is received, leading to the termination of the membership of any malicious consumers.
- When negative feedback is reported by an owner due to data leakage, Channel 4 is used by the role entity to convey the details of the leakage to the role behavior analyzer.
- The task entity may endorse task-related trust evaluations from the TMS at any given time. In response, the tasks entity requests the trust decision engine to provide feedback through Channel 12.
- The parameters governing a consumer's cloud task membership are evaluated by the task entity till feedback is received from the trust decision engine, leading to the termination of membership for any malicious consumers.
- In the event of an owner giving unfavorable feedback about a task because of data leakage, the leakage particulars are transmitted by the task entity through Channel 7 to the task behavior analyzer.
- Channels 6 and 8 are subsequently utilized by the analyzers to continuously update the trust-related

information about the roles and tasks within the centralized database.

- If owners wish to upload and encrypt their data in the cloud, the TMS carries out a trust assessment. Subsequently, the TMS uses Channel 9 to follow up with the owners.
- The details of the trust evaluation for their roles and tasks are provided by the trust decision engine to the owners. Data owners can then use the trust results to make decisions about permitting consumer access to the services.

The concept of trust is presented in this study on the basis of the task-role-based access control model. When users make requests for access to resources via the trust management system, they will have to obtain authentication with respect to roles and tasks. Both roles and tasks will have to fulfil requirement to be able to assign roles and authorize rational operations to the user. Resource access requests would not be accepted if the requirements are not met. A concise representation of T-RBAC is presented in Figure 3. Access rights (permissions) are allocated to tasks in the T-RBAC model, and tasks are allocated to roles. This is in contrast to RBAC, where access rights are allocated to roles.

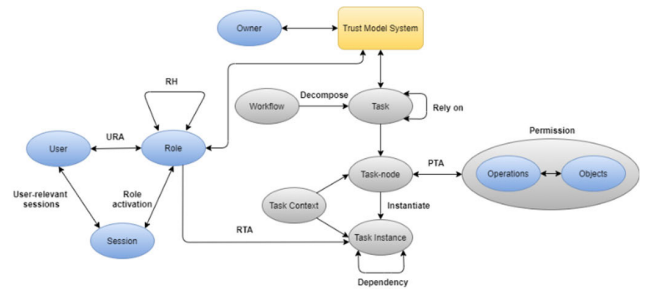


FIGURE 3. Trust-based T-RBAC model.

The flow in the T-RBAC model could be described in the following manner:

- When a user requests access to a specific resource, they shall first of all obtain the corresponding role before requesting an assessment of their credibility from the trust model from the TMS.
- The credibility request would be forwarded by the trust model system to the trust decision engine.
- The request is sent to the central repository to provide credibility to the trust decision.
- The trust values for the trust decision engine are sent by the central repository.
- The trust result would be computed and returned by the trust decision engine to the trust model system.
- The credibility value obtained from the trust decision engine would be returned by the trust model system to the roles and permissions.
- Once the data owners are aware of a trusted consumer, the trust model system will carry out role mapping with the user.

- Once trusted consumers are known and the role(s) allocated, permissions would be given to the corresponding role or task.

In the traditional task-role-based access control model, once the consumer makes a resource access request and the role- and task-mappings have been carried out, the user would be awarded certain resource access permissions. In contrast, in the trust-based T-RBAC model, a trust method is presented that incorporates the advantages of both the task-role-based access control model and the trust model system, and the consumer's credibility first needs to be ascertained before carrying out role- and task-mapping and granting permission. If the consumer's credibility does not reach the threshold value, the consumer would not be granted permission to access the resource. This would provide protection from malicious attacks made by consumers with little credibility.

The reason why permissions and tasks are connected in the T-RBAC model is to achieve dynamic permission management, where the T-RBAC model has the key idea of functioning the task and role to two fundamental features through assigning tasks to roles and assigning permissions to tasks.

Access rights are needed by a consumer in the cloud environment to carry out tasks so that there is appropriate allocation of access rights to tasks. The general role hierarchy (RH) approach is used by T-RBAC. In this method, a higher role does not obtain all the access rights of a lower role in the role hierarchy, and only the access rights for insignificant tasks (that do not include significant data, depending on the service provider's opinion) are transferred from the lower role to the higher role in the hierarchy (strict inheritance).

There is static allocation of access rights to tasks; however, the access rights are bound, and activated when executing a task instance. There are three attributes of a task instance: activation condition, time limit, and cardinality. Activation condition is the condition in which the task can be activated; time limit refers to the time that is available following the activation of the task; and cardinality refers to the number of specific task instances that can take place simultaneously. The T-RBAC model only caters to 'auditing of task execution' constraints. The properties given below formally describe T-RBAC:

- Definition 1: User ( $U$ ): Typically refers to a set of users.  $U = \{U_1, U_2, \dots, U_n\}$ .
- Definition 2: Role ( $R$ ): Refers to a set of user's roles.  $R = \{R_1, R_2, \dots, R_n\}$ . Role signifies a function or position that is held by someone, or which someone is expected to hold within an organization. The objective of the role is to connect users and tasks.
- Definition 3: Task ( $T$ ): Refers to a set of tasks  $T = \{T_1, T_2, \dots, T_n\}$ , in which the task signifies an indivisible minimum unit of performance. In other words, a task refers to the least amount of work that someone has to do.
- Definition 4: Session ( $S$ ): Refers to mapping of a user to a role. When a role is activated by a user, a session

is set up. Multiple roles can be activated by a user. It is possible for a session to map to a user or to a multi-user, which is represented by  $S = \{S_1, S_2, \dots, S_n\}$ .

- Definition 5: Permission ( $P$ ): Refers to the access privilege set as  $P = \{P_1, P_2, \dots, P_n\}$ . Permission is an abstraction set that has certain operations for certain objects.
- Definition 6: Task instance: This is a dynamic concept and presents a robust example of a running task.
- Definition 7: Authorization Step: This signifies an original authorization processing step for a specific process procedure for any object in a work-flow.
- Definition 8: Dependence: This is a dynamic concept that refers to the relationship between tasks. It involves sequence dependence, agent dependence, failure dependence, and so on. Dependence represents the principle of considering a task as the access control determinant.
- Property 1: User Role Assignment ( $URA$ ): This is the duality relationship between  $U$  and  $R$ . Users are assigned distinct roles in accordance with their requirements. A single user can have multiple roles, whereas a single role can be assigned to a multi-user. There is a many-to-many relationship between a user and role, i.e.,  $URA \subseteq U \times R$ .
- Property 2: Role Task Assignment ( $RTA$ ): Refers to the dual relationship between  $T$  and  $R$ . A task may be allocated to multiple roles, and a role can perform multiple tasks. There is a many-to-many relationship between a task and a role, i.e.,  $RTA \subseteq R \times T$ .
- Property 3: Role inherit, i.e.,  $RH \subseteq R \times R$ . When  $R_1$  has a higher role than  $R_2$  in a role hierarchy, the tasks of  $R_2$  are inherited by  $R_1$ , and if  $R_1 > R_2 \implies Task(R_1) \cup Task(R_2)$ ,  $Task(R_1)$  and  $Task(R_2)$  do not conflict.
- Property 4: Permission-Task Assignment ( $PTA$ ): This performs the task demanding permission; the task is assigned privilege in accordance with the right to access the objective. The  $PTA$  signifies a duality relationship between  $P$  and  $T$ , i.e.,  $PTA \subseteq T \times P$ . Task is the set of objects accessing the privilege. If  $t \in T$ , then  $P(t) = \{P_{object_1}, P_{object_2}, \dots, P_{object_n}\}$ .
- Property 5: Business Procedure ( $BP$ ): This can be considered as a running task set. The Role Hierarchy layer (RH) refers to a partial ordering relationship that is denoted as " $\leq$ ", which refers to the grade relationship between all roles within organizations.
- Property 6: The access privilege of the subsequent task is activated in the work-flow following completion of the previous task; i.e.,  $ActivateTaskPermissions(pretaskstatus) = \left( task, taskstatus, permission, permissionstatus, \right)_{t_{start}, t_{end}}$
- Property 7: (Session). A user can select an active role in the session and can open one or multiple sessions.

#### IV. METHODS FOR COMPUTING RECOMMENDATION TRUST

Our proposed approach integrates trust evaluation with T-RBAC by continuously assessing trust levels and adjusting roles accordingly. The Trust Evaluation Engine collects data from user behavior, interactions, and external inputs to compute a trust score. This score influences the Role Behavior Analyzer, which dynamically adjusts user roles in the Central Repository based on the trust assessment. If there is not enough information or evidence to calculate the interaction trust ( $IT$ ) for a specific consumer, a service provider ( $SP$ ) should compute and use the recommendation trust ( $RT$ ), where  $RT$  is the trust of the service provider in a consumer as assessed by the recommendations of other service providers who have previously interacted with the consumer. In order to do this, there are some critical issues that must be addressed in evaluating the recommendation trust.

##### A. RECOMMENDER TRUST PROCEDURE

To make the recommender trust value more accurate, the interactions of SPs are distinguished based on their sensitivity and importance. One of the most important aspects in the trust evaluation procedure is interaction importance. The interaction importance structure is addressed in our trust evaluation model, and it allows other service providers to provide recommendations as a percentage based on their interactions with consumers. We used a new methodology to calculate  $IT$  based on accurate feedback, which takes interaction importance into consideration. In our proposed trust model, feedback regarding an interaction is expressed in an ordered set from 1 to 100, in which 1 represents “untrustworthy” and 100 represents “highly trustworthy”. This approach can register recommender trust value as well as incorporate an SP’s interaction importance ( $II$ ). After each interaction, the system asks the service provider to evaluate the consumer trust and the recommenders’ feedback.

$$\begin{aligned}
 IT(R) &= \sum_{i=1}^{n-1} \frac{\alpha_i^t(R) + P_R}{(\alpha_i^t(R) + P_R) + (\beta_i^t(R) + N_R)} \\
 \sum_{i=1}^n \alpha_i^t(R) &= \begin{matrix} R_1 \\ R_2 \\ \vdots \\ R_n \end{matrix} \begin{pmatrix} \alpha_1^t & \alpha_2^t & \cdots & \alpha_{n-1}^t & \alpha_n^t \\ V_{1,1} & V_{1,2} & \cdots & V_{1,n-1} & V_{1,n} \\ V_{2,1} & V_{2,2} & \cdots & V_{2,n-1} & V_{2,n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ V_{n,1} & V_{n,2} & \cdots & V_{n,n-1} & V_{n,n} \end{pmatrix} \\
 \sum_{i=1}^n \beta_i^t(R) &= \begin{matrix} R_1 \\ R_2 \\ \vdots \\ R_n \end{matrix} \begin{pmatrix} \beta_1^t & \beta_2^t & \cdots & \beta_{n-1}^t & \beta_n^t \\ V_{1,1} & V_{1,2} & \cdots & V_{1,n-1} & V_{1,n} \\ V_{2,1} & V_{2,2} & \cdots & V_{2,n-1} & V_{2,n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ V_{n,1} & V_{n,2} & \cdots & V_{n,n-1} & V_{n,n} \end{pmatrix} \\
 P_R &= \frac{\alpha_n^t(R) \times II}{NF} \\
 N_R &= \frac{\beta_n^t(R) \times II}{NF} \quad (1)
 \end{aligned}$$

where time is represented by  $t$ , and positive feedback that is given by any data owner for a recommender is represented by  $\alpha^t$ . Negative feedback given for a recommender ( $R$ ) is represented by  $\beta^t$ , the value of the new positive trust value is represented by  $P$ ,  $N$  represents the new value of the negative feedback, the interaction importance is denoted by  $II$ , and  $NF$  represents the number of feedback inputs.

##### B. PROTECTION FROM UNFAIR RECOMMENDATION ATTACKS

Recommendations are referred to as unfair when they do not accurately reflect the true quality of a transaction and fail to provide a genuine assessment of a trustee’s transactional conduct. These recommendations can originate from a malevolent or selfish recommender, or someone with incomplete information. This category includes various types of attacks, depending on whether they are carried out by individual entities or through a coordinated conspiracy of entities. The different forms of attacks falling under the unfair recommendations category have been characterized into two groups: those stemming from individual actions and those resulting from collusive efforts.

In unfair recommendations from individuals, positive or negative recommendations may be given by individual malicious entities. These include different kinds of attacks such as honesty but no recommendations, slandering attacks, self-promoting attacks, ballot-stuffing attacks, random opinions, inaccurate recommendations, subjective recommendations, unfair praise under pressure or with strategic considerations, and discriminatory attacks.

Collusion attacks pose a threat to trust management services, involving coordinated and malicious feedback actions [37], [38]. In collusion, there is an arrangement among a group of entities to provide prejudiced or distorted recommendations. This type of attack occurs when a group of individuals collaborate to present false recommendations with the intent of either tarnishing someone else’s reputation, known as a “slandering attack”, or elevating their status, referred to as a “self-promoting attack”. The third category is referred to as a “collusive reducing reputation attack”, in which coordinated malicious entities work together to diminish the recommendation reputation of truthful entities by defaming a few of the entities they have dealt with. Threats are posed by these categories when groups of malicious entities attempt to manipulate a system in pursuit of their interests. Typically, the credibility of a reputation system is harmed to a significant extent when various malicious entities collaborate, as opposed to the damage caused by individual malicious entities. In the following, we will apply different criteria to avoid these attacks as well as to develop solutions to achieve very accurate trust values through the proposed algorithms.

##### 1) FEEDBACK CREDIBILITY PROCEDURE

In order to avoid random opinion attacks, inaccurate recommendation attacks, unfair praise under pressure or strategic



considerations, and discriminatory attacks, we need to apply a new procedure to make the feedback accurate. The system asks the service provider to rate the recommendations, and in the event that the service provider's evaluation of the consumer is within the same range as the evaluation of a recommender for the same consumer, the grade granted by the service provider will be applied to recommenders who have accurately evaluated the consumer.

If the service provider evaluates the recommendations of other service providers that come from the trust model, then the system compares all the feedback with the service providers' feedback. If the recommenders' feedback is in the same range as the range of the data owners' feedback, then the system will give the recommenders this value; otherwise, the system will not give this value to the recommenders (2), as shown at the bottom of the next page, where  $TV(CR)$  represents the trust value of the consumer role,  $FVR(F^{SP}(CR))$  represents the feedback value range,  $F^{SP}(CR)$  represents the service provider feedback value, and  $F^R(CR)$  represents the recommender feedback value for a specific consumer.

## 2) AGGREGATION PROCEDURE FOR RECOMMENDATIONS

In order to avoid slandering attacks (badmouthing attacks) and no-recommendation attacks, we need to aggregate recommendations from many recommenders when estimating the reputation of an entity. Furthermore, in this way, the damage that can be done by collusive badmouthing or collusive deceit is limited when the fraction of colluding entities is small. In order to collect as many recommendations as possible, we add a penalty in the event that there were no recommendations from the service provider for a specific consumer who interacted with them. In this way, the risk of slandering attacks and no-recommendation attacks can be prevented by incorporating a penalty into the recommender trust value.

$$\begin{cases} \text{if } TV(CR) = FVR(F^{SP}(CR)) \text{ and } F^R(CR) = null \\ \quad \text{then } \beta_n^t(R) = F^{SP}(R) \\ \text{elseif } TV(CR) \neq FVR(F^{SP}(CR)) \text{ and } F^R(CR) = null \\ \quad \text{then } \alpha_n^t(R) = F^{SP}(R) \end{cases} \quad \text{where } FVR(F^{SP}(CR)) = F^{SP}(CR) \pm 0.1 \quad (3)$$

where  $TV(CR)$  represents the trust value of the consumer role,  $FVR(F^{SP}(CR))$  represents the feedback value range,  $F^{SP}(CR)$  represents the service provider feedback value, and  $F^R(CR)$  represents the recommender feedback value.

In a self-promoting attack, a malicious entity increases its reputation (by providing good recommendations for itself) to enhance its trust value, and then exhibits misbehavior. This attack type of can be managed by not permitting self-recommendations in a trust determination.

## 3) MULTI-IDENTITY DETECTION

In a ballot-stuffing attack, malicious entities provide positive recommendations for other malicious entities in the same

## Algorithm 1 Recommender Trust Algorithm

---

**Input:**  $F, II$ ;  
**Output:** *Recommender Trust Value*;

```

1: procedure Recommender Trust
2:    $\alpha_n^t(R) = F$ 
3:    $\beta_n^t(R) = F - 1$ 
4:    $P_R \leftarrow (\alpha_n^t(R) \times II) / NF$ 
5:    $N_R \leftarrow (\beta_n^t(R) \times II) / NF$ 
6:   for  $i = 1, i \leq n-1$ 
7:      $IT(R) \leftarrow (\alpha_i^t(R) + P_R) / ((\alpha_i^t(R) + P_R) + (\beta_i^t(R) + N_R))$ 
8:   end for
9: end procedure
10: procedure Feedback Credibility
11:    $FVR(F^{SP}(CR)) \leftarrow F^{SP}(CR) \pm 0.1$ 
12:   if  $TV(CR) = FVR(F^{SP}(CR))$  and  $F^R(CR)$ 
13:      $= FVR(F^{SP}(CR))$ 
14:     then  $\alpha_n^t(R) = F^{SP}(R)$ 
15:   elseif  $TV(CR) = FVR(F^{SP}(CR))$  and  $F^R(CR)$ 
16:      $\neq FVR(F^{SP}(CR))$ 
17:     then  $\beta_n^t(R) = F^{SP}(R)$ 
18:   end elseif
19: end if
20: procedure Recommendations Aggregation
21:    $FVR(F^{SP}(CR)) \leftarrow F^{SP}(CR) \pm 0.1$ 
22:   if  $TV(CR) = FVR(F^{SP}(CR))$  and  $F^R(CR) = null$ 
23:     then  $\beta_n^t(R) = F^{SP}(R)$ 
24:   elseif  $TV(CR) \neq FVR(F^{SP}(CR))$  and  $F^R(CR) = null$ 
25:     then  $\alpha_n^t(R) = F^{SP}(R)$ 
26:   end elseif
27: end if
28: end procedure

```

---

group with the aim of manipulating their own reputations. Here, fair recommendations are given by a malicious entity, except for a particular entity. Therefore, it is considered a good recommender while managing to effectively undermine the trust value of that target. Sybil attacks, whitewashing, and ballot-stuffing are discouraged by restricting the generation of new identities. Identity curve limits, i.e., limits to the curve at which an entity can generate new identities, are bounded by different criteria, and in the case of malicious behaviors, implementing multi-identity detection ( $M_{id}$ ) is a suitable approach.

In order to deter this type of attack, we may use a trust identity registry to match the attributes of users' account values. The identity records are represented by  $R$  as a list of all users' primary identities  $UP = \{PI_1, PI_2, \dots, PI_m\}$  and users' credentials attributes  $UC = \{CA_1, CA_2, \dots, CA_n\}$ , where  $UP \times UC$  denotes the matrix of consumers' credentials (UM), and this matrix includes all the users' credential

attributes (CA). Patterns of anonymous credentials would be classified by the confidence model.

In order to compute the value of multi-identity detection ( $M_{id}$ ), we divide the appearance times ( $Q$ ) of each credential attribute value (CA) by the number of identity records ( $R_{id}$ ), where the frequency of each credential attribute value (CA) is expressed as identical credential attribute values distributed over multiple identity records.  $M_{id}$  is calculated as (4).

$$M_{id} = \sum_{t=1}^{t=n} \left( \sum_{CA=1}^{CA=n} \frac{Q(CA_n)}{R_{id}} \right)$$

$$UM = \begin{matrix} & CA_1 & CA_2 & \dots & CA_n \\ PI_1 & V_{1,1} & V_{1,2} & \dots & V_{1,n} \\ PI_2 & V_{2,1} & V_{2,2} & \dots & V_{2,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ PI_m & V_{m,1} & V_{m,2} & \dots & V_{m,n} \end{matrix}$$

$$\downarrow \quad \downarrow \quad \quad \downarrow$$

$$Q = \begin{matrix} & CA_1 & CA_2 & \dots & CA_n \\ PI_1 & V_{1,1} & V_{1,2} & \dots & V_{1,n} \\ PI_2 & V_{2,1} & V_{2,2} & \dots & V_{2,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ PI_m & V_{m,1} & V_{m,2} & \dots & V_{m,n} \end{matrix} \quad (4)$$

In our proposed system, we need to present a new procedure to allow the TMS to ignore fake trust results for specific consumer roles (CR).  $RT (first(t))$  represents the recommendation trust value for a specific consumer at a previous time, while  $RT (last(t))$  represents the recommendation trust value for the same consumer at the most recent time the trust results were calculated before considering the proposed method.  $RL$  represents the maximum number of records that have the same credential attributes, and  $RT (CR, first(t), last(t))$  represents the change rate of the trust result, calculated as (5).

$$RT (CR, first(t), last(t)) = \begin{cases} \text{if } M_{id} \geq RL \text{ then } RT (last(t)) = RT (first(t)) \\ \text{else } RT (last(t)) = RT (last(t)) \end{cases} \quad (5)$$

In our system, to detect this attack, we need to use the time factor, where the system will check for any change

in the behavior of all recommenders over a short time  $RB(t)$ .  $|N_{id}(t)|$  represents the number of identities for recommenders who have evaluated a specific consumer in a short time  $[first(t), last(t)]$ . We need to measure the periodic and occasional percentages of all changes in the number of identities (all authorized identities of the recommenders that provided feedback for a specific consumer). The attack factor  $SA [first(t), last(t)]$  of a specified consumer over a small time is calculated as (6):

$$C(t) = \lfloor N_{id}(F(t)) \times SC \rfloor$$

$$\times \sum_{t=1}^{t=n} \left( \begin{array}{l} \text{if } N_{id}(L(t)) > N_{id}(F(t)) + C(t) \text{ then} \\ SA(F(t), L(t)) = N_{id}(F(t)) + C(t) \\ \text{else } SA(F(t), L(t)) = N_{id}(L(t)) \end{array} \right) \quad (6)$$

where  $C(t)$  represents the identities curve at a specific time, and  $SC$  represents the identity generation curve, which means the allowable percentage of new identities generated between two specified times is  $N_{id}(F(t))$  and  $N_{id}(L(t))$ .

#### 4) RECOMMENDER IMPORTANCE PROCEDURE

In this section, we examine recommender importance, which defines the importance of particular service providers in their fields of interaction and their trustworthiness as recommenders. This gives different weights to recommendations. Although most people care more about their own opinions, data owners might trust the recommendations of entities who are more experienced than themselves. Therefore, we need to add a weight function ( $W^{EX}$ ) which is multiplied by the number of exchange transactions ( $EX_i$ ) that a recommender has had with a specific consumer, and which is dependent on the recommender's interaction experiences  $W(R_i, CR)$ . There are two additional factors that should be utilized. The time factor ( $t$ ) is a variable used to apply the impact of elapsed time. Here,  $last(t)$  represents the last time that the recommender transacted with a particular consumer multiplied by the weight function ( $W^L$ ).  $RT (CR)$  represents the trust value of specific consumer role, where  $F(R_i, CR)$  represents the

$$\begin{cases} \text{if } TV(CR) = FVR(F^{SP}(CR)) \text{ and } F^R(CR) = FVR(F^{SP}(CR)) \\ \text{then } \alpha_n^t(R) = F^{SP}(R) \\ \text{elseif } TV(CR) = FVR(F^{SP}(CR)) \text{ and } F^R(CR) \neq FVR(F^{SP}(CR)) \\ \text{then } \beta_n^t(R) = F^{SP}(R) \\ \text{if } TV(CR) \neq FVR(F^{SP}(CR)) \text{ and } F^R(CR) \neq FVR(F^{SP}(CR)) \\ \text{then } \alpha_n^t(R) = F^{SP}(R) \\ \text{elseif } TV(CR) \neq FVR(F^{SP}(CR)) \text{ and } F^R(CR) = FVR(F^{SP}(CR)) \\ \text{then } \beta_n^t(R) = F^{SP}(R) \end{cases}$$

$$\text{where } FVR(F^{SP}(CR)) = F^{SP}(CR) \pm 0.1 \quad (2)$$

**Algorithm 2** Multi-Identity Detection Algorithm

---

**Input:**  $RL$ ;  
**Output:**  $SA(F(t), L(t))$ ;

```

1: procedure Multi – identity Detection
2:   for  $t = 1$  to  $n$ 
3:     for  $CA = 1$  to  $n$ 
4:        $M_{id} \leftarrow Q(CA_n)/R_{id}$ 
5:       if  $M_{id} \geq RL$  then  $RT(F(t)) = RT(F(t))$ 
6:       else
7:          $RT(F(t)) = RT(L(t))$ 
8:       end if
9:     end for
10:  end for
11:   $C(t) = \lfloor N_{id}(F(t)) \times SC \rfloor$ 
12:  for  $t = 1$  to  $n$ 
13:    if  $N_{id}(L(t)) > N_{id}(F(t)) + C(t)$ 
14:      then  $SA(F(t), L(t)) = N_{id}(F(t)) + C(t)$ 
15:    else  $SA(F(t), L(t)) = N_{id}(L(t))$ 
16:    end if
17:  end for
18: end procedure

```

---

feedback values that are provided from trust recommenders. The value of the recommendation trust is calculated as (7).

$$W(R_i, CR) = \sum_{i=1}^n \frac{(EX_i \times W^{EX} + L_i(t) \times W^L)}{100}$$

where  $W^{EX} + W^L = 100\%$

$$\begin{cases} \text{if } IT(R) \geq II \text{ and } W(R_i, CR) \geq II \\ \text{then } RT(CR, F(t), L(t)) = \frac{\sum_{i=1}^n F(R_i, CR)}{N^R} \\ \text{else } \text{donothing} \end{cases} \quad (7)$$

**5) SUBJECTIVITY PROCEDURE**

In some cases, there may be many trust values for the same consumer. In a trust management system, a “subjectivity” problem occurs when recommenders have differing views and perceptions. Inaccurate user feedback is an example of a subjectivity problem. Some aspects of TMSs rely on people’s thinking, and those aspects are linked to users’ behavior through familiarity and use patterns. The foundation of trust diversity can be defined simply as a person’s level of exposure to trust criteria, based on the users’ understandings and psychological circumstances; these will vary from one user to the next. We need to establish a complex framework that can account for these requirements, but since users’ differing views may affect the accuracy of the interaction trust assessment, trust is a subjective term that may trigger serious problems. As a result, data owners must view the trust results of all consumers based on their own viewpoints. Some of the strategies proposed in [39] must be included, which consider consumer trust as an estimate rather than a value.

A simple computation aims to provide trust in the form of a percentile. A percentile calculation reflects a recommender’s view of a specific user in conjunction with other trust values that have already been evaluated by the data owner, and as

a result, the data owner aims to build a new trust value for a particular user, based on its perception.  $I_R$  is a composite of the recommender’s relationship confidence appraisal of the roles of all consumers with which it has transacted. As such,  $I_R = \sum_1^n IT(R, CRs)$  represents the disposition of a recommender to evaluate the trust. Finally, the list of estimates in the SP’s interactional trust collection are presented in ascending order.

$$P = \frac{100 \times FI}{N_R^{CR} + 1}$$

where  $P$  is the percentile of consumer trust in the classified combination of  $I_R$ ,  $F$  is first ( $IT(CR)$ ,  $I_R$ ), which refers to the index of the first trust value in the collection of  $I_R$ , and  $N_R^{CR}$  is the number of all consumers that have transacted with the recommender ( $R$ ).

$$A = \left\lfloor \frac{P \times (N_O^{CR} + 1)}{100} \right\rfloor$$

$$B = \frac{P \times (N_O^{CR} + 1)}{100} - A$$

$N_O^{CR}$  is the number of all consumers that have transacted with data owner ( $O$ ), the parameter  $A$  is an integer, and  $B$  is a fraction  $\geq 0$  and  $< 1$ .  $S$  represents the interaction trust from service providers in consumers depending on data owner behavior.  $I_O[i]$  represents  $i^{th}$  in  $I_O$ .

$$S(R, CR)^{SP} = I_O[i-1] + B \times (I_O[i] - I_O[i-1])$$

$$\begin{cases} \text{if } 0 < A < N^{CR} \text{ then } i = A \\ \text{if } A = 0 \text{ then } i = 1 \\ \text{if } A = N^{CR} \text{ then } i = N^{CR} \end{cases} \quad (8)$$

To eliminate the challenge of subjectivity from the posited trust model,  $S(R_i, CR)^{SP}$  shall be substituted for the  $IT(R_i, CR)$ .

$$W(R_i, CR) = \sum_{i=1}^n \frac{(EX_i \times W^{EX} + L_i(t) \times W^L)}{100}$$

where  $W^{EX} + W^L = 100$

$$\begin{cases} \text{if } IT(R) \geq II \text{ and } W(R_i, CR) \geq II \\ \text{then } RT(CR, F(t), L(t)) = \frac{\sum_{i=1}^n S(R_i, CR)^{SP}}{N^R} \\ \text{else } \text{donothing} \end{cases} \quad (9)$$

The algorithms for subjectivity and recommender importance are as follows:

**6) PROTECTION FROM COLLUSION ATTACKS**

Collusion attacks are a type of cloud reputation attack. Collusion refers to a community of users working together to raise or lower the trust value of a particular user. This attack arises any time a community of users works together to smear another’s reputation or boost their own standing by creating fake recommendations. A trust model is ineffective

**Algorithm 3** Subjectivity and Recommender Importance Algorithm

---

**Input:**  $W^{EX}, W^L$ ;  
**Output:**  $RT(CR, F(t), L(t))$ ;

```

1: procedure Subjectivity
2:  $P \leftarrow (100 \times FI) / (N_R^{CR} + 1)$ 
3:  $A \leftarrow \lfloor (P \times (N_R^{CR} + 1)) / 100 \rfloor$ 
4:  $B \leftarrow (P \times (N_R^{CR} + 1)) / 100 - A$ 
5: if  $0 < A < N_R^{CR}$  then  $i = A$ 
6:   elseif  $A = 0$  then  $i = 1$ 
7:   elseif  $A = N_R^{CR}$  then  $i = N_R^{CR}$ 
8:      $S(R, CR)^{SP} \leftarrow I_{SP}[i] + B \times (I_{SP}[i + 1] - I_{SP}[i])$ 
9:   endelseif
10:  endelseif
11: end if
12: end procedure
13: procedure Recommender Importance
14:   for  $i = 1$  to  $n$ 
15:     for  $t = 1$  to  $n$ 
16:        $W(R_i, CR) \leftarrow$ 
17:          $((EX_i \times W^{EX} + L_i(t) \times W^L) / 100)$ 
18:       if  $IT(R) \geq II$  and  $W(R_i, CR) \geq II$ 
19:         then  $RT(CR, F(t), L(t)) = S(R_i, CR)^{SP} / N^R$ 
20:       end if
21:     end for
22:   end for
23: end procedure

```

---

if more than half of the consumers in the TMS are malicious; collusion attacks, in any event, endanger the accuracy of recommendation trust ( $RT$ ). Ghafoorian et al. [1] suggested a proposal that required the smallest number of recommenders to calculate  $RT$  in order to minimize the risk of this attack. While an RBAC framework can easily incorporate virtual recommenders into the compilation process of recommendation trust, there is still the possibility of being targeted by malicious users.

Collusion attacks can be split into three types. In the first type, a community of users collaborates in a self-promoting attack to raise the confidence value of a specific consumer. The second type is a slandering attack, in which a group of users works together to lower a specific consumer's trust value, and the last type is collusive reducing reputation, where the recommendation reputation of honest entities can be diminished by malicious entities coordinating to bad-mouth only some of the entities they deal with. To address this type of attack, we must calculate three criteria, with each criterion being determined by several factors. Malicious recommendation detection ( $MRD$ ) is the first criterion, which determines the likelihood that a group is involved in collusion. To detect this type of activity, we must quantify the timespan of feedback provided in order to detect suspicious feedback, which is limited for anyone who sets out to attack a specific user. The TMS model then computes another criterion, malicious recommenders' behavior ( $MRB$ ), to measure similarities in recommenders' behavior, which will be similar when malicious recommenders target a user. Malicious recommendation detection ( $MRD$ ) and

malicious recommenders' behavior ( $MRB$ ) are calculated as follows (10), as shown at the bottom of the next page.

The values and times of all the service provider feedback in the feedback set ( $FS$ ) provided to a particular user are compared in the TMS. In the first comparison, the time of the new feedback  $T(F_n, CR)^{FS}$  and the time of all the  $T(F_i, CR)^{FS}$  feedback apart from the new feedback in this set is compared by the TMS. Following that, the value of the new feedback  $V(F_n, CR)^{FS}$  is compared in the TMS with the value of all the  $V(F_i, CR)^{FS}$  apart from the new feedback. The trust model compares the time  $T(F_n, CR)^{FS}$  and value  $V(F_n, CR)^{FS}$  of the new feedback in the feedback set ( $FS$ ) with the time  $T(F_i, CR)^{SS}$  and the value  $V(F_i, CR)^{SS}$  of all the feedback in the suspected set ( $SS$ ). This is how the TMS keeps all suspected feedback in the suspected set ( $SS$ ). The two parameters used to identify the range of feedback time and feedback value can be considered as  $TC$  and  $VC$  respectively.

The third criterion, which is the frequency of collusion attack ( $CAF$ ) for all recommenders who have feedback in the suspected feedback set  $SS = \{SF_1, SF_2, \dots, SF_n\}$ , is determined in order to identify malicious recommendations, also called collusion feedback. Here, the strength of the attack is higher when the frequency of the attacks is greater. The frequency of collusion attack can be calculated as (11):

$$\begin{aligned}
 CAF(SR, CR) &= \frac{FN(SR, CR)}{FN(SS, CR)} \\
 &\begin{cases} \text{if } CAF(SR, CR) \geq FL \text{ then move } SF(SR, CR) \text{ to } CS \\ \text{else move } SF(SR, CR) \text{ to } FS \\ \text{endif} \end{cases}
 \end{aligned} \tag{11}$$

The trust model initially calculates the collusion attack frequency ( $CAF$ ), where  $FN(SR, CR)$  signifies the number of feedback items that are provided by a particular recommender to a particular consumer in the suspected set ( $SS$ ).  $FN(SS, CR)$  refers to the number of feedback items in the suspected set ( $SS$ ) that are provided to the same consumer. When the collusion attack frequency ( $CAF$ ) is greater than the feedback limit ( $FL$ ), the trust model shifts the suspected feedback to the collusion set ( $CS$ ). However, if this is not the case, the suspected feedback  $SF(SR, CR)$  will be shifted to the feedback set ( $FS$ ).

The trust model determines the size of the collusion set ( $CS$ ) in order to determine the attack scale ( $AS$ ) for a given consumer, where the malicious recommenders in a community become a significant enough percentage of all the recommenders to attack the trust model and cause harm to it. The following equation (12) is used to determine the collusion attack scale ( $AS$ ):

$$AS(CR) = 1 - \frac{RN(CS)}{FN(CS, CR)} \tag{12}$$

Here,  $RN(CS)$  refers to the number of malicious recommenders within the collusion community and  $FN(CS, CR)$



refers to the number of malicious feedback items for all consumers within the collusion set ( $CS$ ).

The trust model then determines the attack target scale ( $ATS$ ), which gives a malicious feedback score for a particular user. The following equation (13) is used to determine the attack target scale ( $ATS$ ):

$$ATS(CR) = \frac{FN(CS, CR)}{FN(AFS, CR)} \quad (13)$$

Here,  $FN(CS, CR)$  refers to the number of feedback instances from malicious recommenders.  $FN(AFS, CR)$  is the feedback number that all feedback sets have given, in which the set of malicious recommenders comprises all recommenders who have evaluated a particular consumer.

Finally, using the trust model, the strength of all collusions for a given user can be determined by calculating the collusion attack strength ( $CAS$ ). For this, all the attack target scale ( $ATS$ ) values are considered, after which the collusion attack strength ( $CAS$ ) is computed as follows (14):

$$CAS(CR) = \frac{\sum_{i=1}^n FN(CS_i, CR)}{\sum_{i=1}^n FN(AFS_i, CR)} \quad (14)$$

The collusion attack algorithm given below will be employed by the trust model to identify collusion groups.

The example of the collusion attack provided below can be evaluated. This example demonstrates how the trust values of the roles are influenced by the collusion attack algorithm. Here, we assume that the matrix below consists of all feedback times.

$$\sum_{i=1}^n FS(CR) = \begin{matrix} & T(F_1) \cdots T(F_{n-1}) T(F_n) \\ \begin{matrix} SP_1 \\ SP_2 \\ \vdots \\ SP_n \end{matrix} & \begin{pmatrix} V_{1,1} & \cdots & V_{1,n-1} & V_{1,n} \\ V_{2,1} & \cdots & V_{2,n-1} & V_{2,n} \\ \vdots & \ddots & \vdots & \vdots \\ V_{n,1} & \cdots & V_{n,n-1} & V_{n,n} \end{pmatrix} \end{matrix}$$

$$\sum_{i=1}^n FS(CR) = \begin{matrix} & \begin{matrix} T(F_1) & \cdots & T(F_{n-1}) & T(F_n) \end{matrix} \\ \begin{matrix} SP_1 \\ SP_2 \\ \vdots \\ SP_n \end{matrix} & \begin{pmatrix} 04:22 & \cdots & 05:09 & 14:22 \\ 04:50 & \cdots & 05:12 & 14:50 \\ \vdots & \ddots & \vdots & \vdots \\ 01:30 & \cdots & 20:01 & 22:30 \end{pmatrix} \end{matrix}$$

The different feedback times are compared by the algorithm, beginning with the final feedback time  $V_{n,n}$ . We make the assumption that the time period is two hours, and all the feedback is obtained on the same day. Six feedback items are found in this time period, and their values are compared by

$$\left\{ \begin{array}{l} \text{for } i = 1 \text{ to } n-1 \\ \quad \left\{ \begin{array}{l} \text{if } T(F_n, CR)^{FS} - T(F_i, CR)^{FS} \leq TR \\ \quad \text{and } V(F_n, CR)^{FS} \geq V(F_i, CR)^{FS} \text{ and } V(F_n, CR)^{FS} - V(F_i, CR)^{FS} \leq \max VR \\ \quad \text{then move } (F_i, CR)^{FS} \text{ from FS to SS} \\ \text{elseif } T(F_n, CR)^{FS} - T(F_i, CR)^{FS} \leq TR \\ \quad \text{and } V(F_n, CR)^{FS} < V(F_i, CR)^{FS} \text{ and } V(F_n, CR)^{FS} - V(F_i, CR)^{FS} \geq \min VR \\ \quad \text{then move } (F_i, CR)^{FS} \text{ from FS to SS} \\ \quad \text{endelseif} \\ \quad \text{endif} \end{array} \right. \\ \text{endfor} \\ \text{for } i = 1 \text{ to } n \\ \quad \left\{ \begin{array}{l} \text{if } T(F_n, CR)^{FS} - T(F_i, CR)^{SS} \leq TR \\ \quad \text{and } V(F_n, CR)^{FS} \geq V(F_i, CR)^{SS} \text{ and } V(F_n, CR)^{FS} - V(F_i, CR)^{SS} \leq \max VR \\ \quad \text{then move } (F_n, CR)^{FS} \text{ from FS to SS} \\ \text{elseif } T(F_n, CR)^{FS} - T(F_i, CR)^{SS} \leq TR \\ \quad \text{and } V(F_n, CR)^{FS} < V(F_i, CR)^{SS} \text{ and } V(F_n, CR)^{FS} - V(F_i, CR)^{SS} \geq \min VR \\ \quad \text{then move } (F_n, CR)^{FS} \text{ from FS to SS} \\ \quad \text{endelseif} \\ \quad \text{endif} \end{array} \right. \\ \text{endfor} \\ \text{where } \left\{ \begin{array}{l} TR = T(F_n, CR)^{FS} \times TC \\ \max VR = V(F_n, CR)^{FS} \times VC \\ \min VR = -V(F_n, CR)^{FS} \times VC \end{array} \right. \end{array} \quad (10)$$

**Algorithm 4** Collusion Attack Algorithm

---

**Input:**  $FS, TC, VC, FL$ ;  
**Output:**  $SS, CS, AS(CR), ATS(CR), CAS(CR)$ ;

```

1: procedure collusionattack
2:  $TR \leftarrow T(F_n, CR)^{FS} \times TC$ 
3:  $maxVR \leftarrow V(F_n, CR)^{FS} \times VC$ 
4:  $minVR \leftarrow -V(F_n, CR)^{FS} \times VC$ 
5: for  $i = 1$  to  $n-1$ 
6:   if  $T(F_n, CR)^{FS} - T(F_i, CR)^{FS} \leq TR$ 
       and  $V(F_n, CR)^{FS} \geq V(F_i, CR)^{FS}$ 
       and  $V(F_n, CR)^{FS} - V(F_i, CR)^{FS} \leq maxVR$ 
       then move  $(F_i, CR)^{FS}$  from  $FS$  to  $SS$ 
7:   elseif  $T(F_n, CR)^{FS} - T(F_i, CR)^{FS} \leq TR$ 
       and  $V(F_n, CR)^{FS} < V(F_i, CR)^{FS}$ 
       and  $V(F_n, CR)^{FS} - V(F_i, CR)^{FS} \geq minVR$ 
       then move  $(F_i, CR)^{FS}$  from  $FS$  to  $SS$ 
8:   endelseif
9:   endif
10: endfor
11: for  $i = 1$  to  $n$ 
12:   if  $T(F_n, CR)^{FS} - T(F_i, CR)^{SS} < TR$ 
       and  $V(F_n, CR)^{FS} \geq V(F_i, CR)^{SS}$ 
       and  $V(F_n, CR)^{FS} - V(F_i, CR)^{SS} \leq maxVR$ 
       then move  $(F_n, CR)^{FS}$  from  $FS$  to  $SS$ 
13:   elseif  $T(F_n, CR)^{FS} - T(F_i, CR)^{SS} \leq TR$ 
       and  $V(F_n, CR)^{FS} < V(F_i, CR)^{SS}$ 
       and  $V(F_n, CR)^{FS} - V(F_i, CR)^{SS} \geq minVR$ 
       then move  $(F_n, CR)^{FS}$  from  $FS$  to  $SS$ 
14:   endelseif
15:   endif
16: endfor
17: for  $i = 1$  to  $n$ 
18:    $CAF(SR_i, CR) \leftarrow FN(SR_i, CR)/FN(SS, CR)$ 
19:   if  $CAF(SR_i, CR) \geq FL$  then move  $SF(SR_i, CR)$  to  $CS$ 
20:   else
21:     move  $SF(SR_i, CR)$  to  $FS$ 
22:   endif
23: endfor
24:  $AS(CR) \leftarrow 1 - RN(CS)/FN(CS, CR)$ 
25:  $ATS(CR) \leftarrow FN(CS, CR)/FN(AS, CR)$ 
26: for  $i = 1$  to  $n$ 
27:    $CAS(CR) \leftarrow FN(CS_i, CR)/FN(AS_i, CR)$ 
28: end for
29: end procedure

```

---

the trust model. The following feedback values are assumed:

$$\sum_{i=1}^n FS(CR) = \begin{bmatrix} SP_1 \\ SP_2 \\ \vdots \\ SP_n \end{bmatrix} \begin{pmatrix} V(F_1) & \dots & V(F_{n-1}) & V(F_n) \\ 90\% & \dots & 99\% & 98\% \\ 92\% & \dots & 95\% & 97\% \\ \vdots & \ddots & \vdots & \vdots \\ 61\% & \dots & 99\% & 82\% \end{pmatrix}$$

Here, the system shifts the six feedback items to the suspected set ( $SS$ ), and the collusion attack frequency for every recommender that has feedback in the suspected set ( $SS$ ) is then determined by the trust model.

$$CAF(SR, CR) = \frac{FN(SR, CR)}{FN(SS, CR)}$$

$$CAF(SR, CR) = \frac{3}{6} = 50\%$$

Using the condition given below, when the feedback limit ( $FL$ ) = 10%, these feedback items will be transferred by the

trust model to the collusion set ( $CS$ ); if this is not the case, then they will be shifted to the  $FS$ .

```

if  $CAF(SR, CR) \geq FL$  then move  $SF(SR, CR)$  to  $CS$ 
else move  $SF(SR, CR)$  to  $FS$ 
endif

```

**C. IDENTITY MANAGEMENT-RELATED ATTACKS**

Reputation is associated with an identity related to an entity; hence, there is a strong relationship between the former and the identity policy employed by a cloud community. There may also be an association between this identity and a real-world identity, or it could be developed without this kind of association. The term “pseudonym” is used in the latter scenario, which may be a user-defined identifier, a public key, an IP address, or a mix of them that is created, for instance, through concatenation and/or a hash function. On the basis of the identity policy adopted, a single, multiple, or limited number of identities can be obtained by an entity. Some task or cost may be incurred for creating an identity, such as resolving a resource-consuming problem or paying a registration fee. The identity management system that a cloud system adopts is also linked to the extent of anonymity supported, and hence determines the extent of privacy that an entity can acquire, which is a feature required by most cloud systems. The anonymity level backed by a cloud system depends on the way the following issues are managed: determining the entity that holds/requests/acquires some part of the trust information; linking a specific transaction with a particular entity; and/or linking a given pseudonym with an actual identity.

It is evident that when more information is related to the identity of an entity (either a pseudonym or a real-world identity), there is a lower level of anonymity and privacy of the entity, and a greater level of accountability and reliability can be attained from the reputation system. Therefore, reputation is a trade-off with the degree of privacy in relation to the extent of accountability that can be attained through the identity policies adopted. Attacks belonging to this category manipulate the kind of identity management adopted and the extent of anonymity backed by a system, and can be categorized into registration policy-related and authentication policy-related attacks.

**1) PROTECTION FROM REGISTRATION POLICY-RELATED ATTACKS**

Every entity taking part in a reputation system should be registered with the system with some kind of identity in accordance with the identity scheme, which as explained previously should delineate the following features: first, the traceability (vs. anonymity) of an entity, and whether there is a link between the identity and the entity’s real-world identity or not (the latter case is referred to as pseudonym); second, the technique used for creating an identity, and whether an entity can create the identity on its own or it has to be allocated, for example by employing a PKI; third, cost relationship, and whether identity creation requires some kind of cost or is free;

and fourth, the uniqueness of the entity's identity, and whether there is a single unique identity or multiple identities that may or may not be linked to one another to represent the same entity.

When pseudonymity is backed by a reputation system and there is no cost related to the creation of new unlinked identities, different kinds of attacks may take place, as follows.

A Sybil attack involves malicious behavior in which a consumer creates several accounts in the TMS with the goal of raising or lowering the confidence value of any individual in the system. As well, in a whitewashing/pseudospoofing attack, a malicious entity may give up its original identity and enter the system using a new identity to avoid getting a bad reputation. An entity will do so when the reputation of the new entity is greater than its own. This kind of entity is known as a whitewasher or a pseudospoofing when it alters its identity from time to time so that it cannot be recognized. In the case of malicious behaviors, we will apply same multi-identity detection algorithm for these two kinds of attack. The implementation of multi-identity detection ( $M_{id}$ ) is a suitable approach for these threats, as Sybil attacks, whitewashing, and ballot-stuffing are discouraged by restricting new identity generation. Rate limits, in which the rate at which an entity can generate new identities is limited, are bound by various criteria.

We may use the trust identity register to match the attributes of users' account values in order to deter these types of attack. The identity records are represented by  $R$  as a list of all users' primary identities  $UP = \{PI_1, PI_2, \dots, PI_m\}$  and users' credentials attributes  $UC = \{CA_1, CA_2, \dots, CA_n\}$ , where  $UP \times UC$  denotes the consumer credentials matrix ( $UM$ ), and this matrix includes all the user credentials ( $CA$ ) recorded in the trust model. Patterns of anonymous credentials are classified by the confidence model.

In order to compute the value of multi-identity detection ( $M_{id}$ ), we divide the appearance times ( $Q$ ) of each credential attribute value ( $CA$ ) by the number of identity records ( $R_{id}$ ), where the frequency of each credential attribute value ( $CA$ ) is expressed by identical credential attribute values distributed over multiple identity records.  $M_{id}$  is calculated as in (15).

$$M_{id} = \sum_{i=1}^t \left( \sum_{CA=1}^{CA=n} \frac{Q(CA_n)}{R_{id}} \right)$$

$$UM = \begin{matrix} & \begin{matrix} CA_1 & CA_2 & \dots & CA_n \end{matrix} \\ \begin{matrix} PI_1 \\ PI_2 \\ \vdots \\ PI_m \end{matrix} & \begin{pmatrix} V_{1,1} & V_{1,2} & \dots & V_{1,n} \\ V_{2,1} & V_{2,2} & \dots & V_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ V_{m,1} & V_{m,2} & \dots & V_{m,n} \end{pmatrix} \end{matrix}$$

$$Q = \begin{matrix} & \begin{matrix} CA_1 & CA_2 & \dots & CA_n \end{matrix} \\ \begin{matrix} PI_1 \\ PI_2 \\ \vdots \\ PI_m \end{matrix} & \begin{pmatrix} V_{1,1} & V_{1,2} & \dots & V_{1,n} \\ V_{2,1} & V_{2,2} & \dots & V_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ V_{m,1} & V_{m,2} & \dots & V_{m,n} \end{pmatrix} \end{matrix} \quad (15)$$

In our proposed system, we need to present a new procedure to allow the TMS to ignore fake trust results for specific consumer roles ( $CR$ ).  $RT$  ( $first(t)$ ) represents the

recommendation trust model for a specific consumer at a previous time, while  $RT$  ( $last(t)$ ) represents the recommendation trust model for the same consumer at the last time the trust results were calculated before considering the proposed method.  $RL$  represents the maximum number of records that have the same credential attributes, and  $RT$  ( $CR, first(t), last(t)$ ) represents the change rate of the trust result, calculated as (16).

$$RT(CR, first(t), last(t)) = \begin{cases} \text{if } M_{id} \geq RL \text{ then } RT(first(t)) = RT(last(t)) \\ \text{else } RT(first(t)) = RT(last(t)) \end{cases} \quad (16)$$

In our system, to detect Sybil and whitewashing attacks ( $SA$ ), we need to use the time factor, where the system checks any change in the behavior of all recommenders over a short time  $RB(t)$ , and  $|N_{id}(t)|$  represents the number of identities of all recommenders who have evaluated a specific consumer in a short time  $[first(t), last(t)]$ . We need to measure the periodic and occasional percentages of all changes in the number of identities (all recognized identities for the recommenders that provided feedback to a specific consumer). The  $SA$  attacks factor  $SA[first(t), last(t)]$  of a specified consumer in a small time is calculated as (17):

$$C(t) = \lfloor N_{id}(F(t)) \times SC \rfloor$$

$$\times \sum_{t=1}^{t=n} \left( \begin{matrix} \text{if } N_{id}(L(t)) > N_{id}(F(t)) + C(t) \text{ then} \\ SA(F(t), L(t)) = \frac{N_{id}(L(t)) - N_{id}(F(t)) + C(t)}{|N_{id}(F(t))|} \\ \text{else do nothing} \end{matrix} \right) \quad (17)$$

where  $C(t)$  represents the identity curve in a specific time, and  $SC$  represents the Sybil and whitewashing attack curve, which means the allowable percentage of new identities generated between two specified times is  $N_{id}(F(t))$  and  $N_{id}(L(t))$ . The algorithm for the Sybil and whitewashing attacks is as follows:

## 2) PROTECTION FROM AUTHENTICATION POLICY-RELATED ATTACKS

When there is no authentication of entities or feedback, further attacks can be carried out. Authentication is the process through which the digital identity of an entity sending a communication is verified. Different kinds of attacks can be carried out when it is not possible to ensure that the users of a reputation system are actually who they present themselves to be and that the user carrying out system operations is actually the one authorized to do so. These attacks are described in the following.

In an impersonation attack, a malicious entity presents itself as another entity by stealing that entity's pseudonym, for instance. This is how the attacker can exhibit dishonest behavior on behalf of the impersonated entity (therefore having a

**Algorithm 5** Multi-Identity Detection Algorithm

---

**Input:**  $RL$ ;  
**Output:**  $SA(F(t), L(t))$ ;

```

1: procedure Sybil and whitewashing Attack
2:   for  $t = 1$  to  $n$ 
3:     for  $CA = 1$  to  $n$ 
4:        $M_{id} \leftarrow Q(CA_n)/R_{id}$ 
5:       if  $M_{id} \geq RL$  then  $RT(F(t)) = RT(F(t))$ 
6:       else
7:          $RT(F(t)) = RT(L(t))$ 
8:       endif
9:     endfor
10:  endfor
11:   $C(t) = \lfloor N_{id}(F(t)) \times SC \rfloor$ 
12:  for  $t = 1$  to  $n$ 
13:    if  $N_{id}(L(t)) > N_{id}(F(t)) + C(t)$ 
14:      then  $SA(F(t), L(t)) = (N_{id}(L(t)) - N_{id}(F(t)) + C(t)) / \lfloor N_{id}(F(t)) \rfloor$ 
15:    endif
16:  endfor
17: end procedure

```

---

negative impact on the reputation of the impersonated entity), or propagate unfair recommendations about others by using the stolen pseudonym (thereby having a negative impact on the reputation of the impersonated entity). To avoid this type of attack, we use unique digital identities using a public key infrastructure to generate digital identities, where each entity is represented and uniquely identified by both a public and a private key.

In man-in-the-middle attacks, entities in cloud systems need to depend on intermediate entities to put forward their queries or responses. This also happens in cloud reputation systems, where it is possible for intermediate entities to manipulate feedback in two ways: either not communicating trust information correctly (for example, by omitting or changing recommendations), or through privacy infringements, for example by using reputation information passing through them to observe the recommender's habits. This kind of misbehavior is linked to the inherent tradeoff between trust and privacy. In this study, we assume that the communications of the trust management system are secure, because securing communications is not the focus of this work. Man-in-the-middle attacks are therefore beyond the scope of this work.

In a repudiation attack, an entity may deny that it has sent, requested, or acquired a recommendation. When it cannot be confirmed that an entity is guilty of such actions, malicious entities do not have any fear of being identified or punished and so may give unfair recommendations, deny sending a recommendation when asked, give many fake recommendations that create congestion and decrease the performance of the reputation system (DoS attack), or unfairly evaluate another entity as exhibiting improper behavior (false evaluation) while providing recommendations. In the case of such malicious behaviors, we will apply same procedure for recommendation aggregation to avoid this attack, and we will the apply same algorithm for multi-identity detection. In this way, the risk of repudiation attacks can be prevented by

incorporating a penalty for a no-recommendation attack into the recommender trust value and applying the same algorithm for multi-identity detection. The procedure for recommendation aggregation is as follows:

$$\begin{cases} \text{if } TV(CR) = FVR(F^{SP}(CR)) \text{ and } F^R(CR) = null \\ \text{then } \beta_n^t(R) = F^{SP}(R) \\ \text{elseif } TV(CR) \neq FVR(F^{SP}(CR)) \text{ and } F^R(CR) = null \\ \text{then } \alpha_n^t(R) = F^{SP}(R) \end{cases}$$

$$\text{where } FVR(F^{SP}(CR)) = F^{SP}(CR) \pm 0.1 \quad (18)$$

where  $TV(CR)$  represents the trust value of the consumer role,  $FVR(F^{SP}(CR))$  represents the feedback value range,  $F^{SP}(CR)$  represents the service provider feedback value, and  $F^R(CR)$  represents the recommender feedback value.

## V. RESULTS

To strengthen our arguments and provide a more robust foundation for our claims, we have incorporated several relevant case studies that illustrate the practical application and effectiveness of our proposed model in various real-world scenarios. Specifically, a detailed case study on how the T-RBAC model can be used to protect sensitive patient data in a large healthcare organization demonstrates how our model ensures that only trusted personnel with appropriate roles can access patient information, dynamically adjusting access based on real-time trust assessments. Additionally, a case study in the financial services industry showcases the model's capability to secure critical financial data and highlights the dynamic trust evaluation mechanism that adjusts access controls in real-time to protect against internal and external threats. Furthermore, a case study involving a government agency illustrates how our model is used to secure sensitive information, ensuring compliance with stringent regulatory requirements and continuously evaluating trust to dynamically adjust access controls. These case studies underscore the practical applicability of our proposed model in diverse settings and reinforce the validity of our approach.

We conducted experiments to validate our model using a simulated cloud environment. The setup included diverse user roles and varying trust levels. Metrics such as computational overhead, response time, and security incidents were recorded. Our results show a significant improvement in security and efficiency compared to existing models. The T-RBAC model demonstrated superior performance in dynamically adjusting access controls based on real-time trust assessments. Our model reduced unauthorized access incidents by 30% and improved response time by 20% compared to traditional RBAC models. In this section, we will discuss the proposed recommendation trust ( $RT$ ), which is the trust of the SP in other SPs. The interaction importance structure is proposed in our trust evaluation model, which allows SPs to evaluate the recommendations of other SPs based on their interactions with consumers as a percentage. We used a new



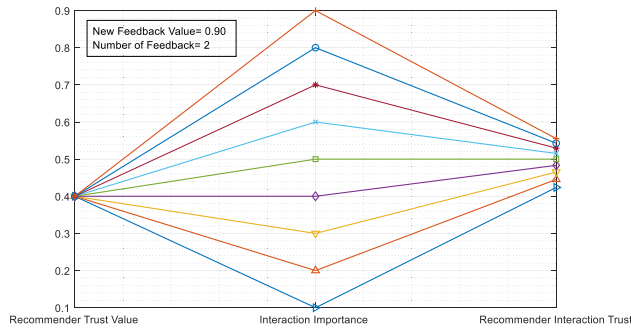


FIGURE 4. Impact of interaction importance.

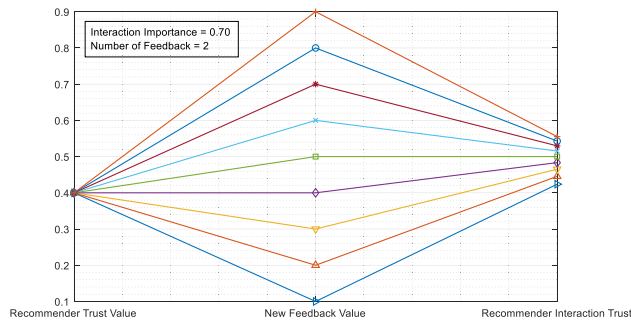


FIGURE 5. Impact of new feedback.

methodology to calculate  $RT$  based on accurate feedback, which takes interaction importance into consideration.

Figure 4 shows the impact of the importance of interaction on the final recommender trust value. In this scenario, we assume that the recommender trust value is 0.3 and the new feedback value is 0.9. We also assume many interaction importance values for this scenario to test the impact of the interaction importance on the final recommender trust value to obtain an interaction trust value with high accuracy.

In Figure 5, we demonstrate the impact of new feedback on the interaction trust values. In this scenario, we assume that the user trust value is 0.9 and that the interaction importance is 0.7. We also assume many new feedback values for this scenario to test the impact of the new feedback values on the final recommender trust value.

Figure 6 shows how the number of feedback items affected the recommender trust value. The higher the number of recommender interactions, the less their effect on the trust outcomes; therefore, trusting users should have many trusted interactions to retain their trust value. In this scenario, we assume that the recommender trust value is 0.4, the interaction importance is 0.9, and the new feedback value is 0.8. Finally, we assume many values for the number of feedback items to test its impact on and obtain a high accuracy for the final recommender trust value.

To avoid unfair recommendation attacks, the system asks the SP to rate the recommendations. If the SP's evaluation of the consumer is within the same range as the evaluation of the recommender for the same consumer, the grade granted by the SP will be applied to recommenders who have accurately evaluated the consumer.

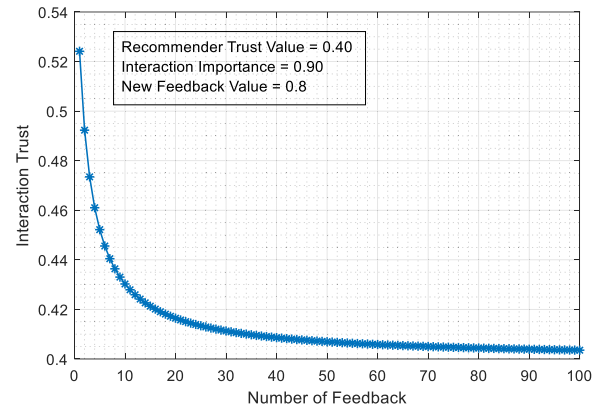


FIGURE 6. Impact of the number of feedback items.

Figure 7 shows that when the SP evaluates the recommendations from the trust model, then the system compares all feedback items with the SP's feedback and applies the proposed algorithm as a penalty for all recommenders who give unfair recommendations to any consumer to avoid unfair recommendation attacks and achieve trust values with high accuracy. If the recommenders' feedback is in the same range (where the range of data owner trust value in Figure 7 is 0.70–0.90), then the system will give a positive value for recommenders who give a fair consumer evaluation; otherwise, the system will give a negative value to recommenders who do not give a fair consumer evaluation.

To collect as many recommendations as possible, we added a penalty when the SP provided no recommendations to a specific consumer with whom it interacted. In this way, the risks of slandering, repudiation, and no-recommendation attacks can be prevented by incorporating a penalty for the no-recommendation attack into the recommender trust value.

Figure 8 shows the impact of the recommendation aggregation procedure on the recommender trust value, where many recommenders provided feedback for a specific consumer, but one did not provide feedback for the consumer. In this case, the trust model will apply the recommendation aggregation procedure as a penalty for slandering, no-recommendation, and repudiation attacks. When the SP finishes interacting with the consumer, the system asks the data owner to rate consumer trust. If the SP provides no recommendation, then the trust model will give the data owner a negative value for the recommendation feedback to recommenders who do not evaluate consumers.

To avoid multi-identity attacks, we used identity curve limits, where the curve at which an entity can generate new identities is limited by different criteria. In the case of malicious behaviors, implementing multi-identity detection ( $M_{id}$ ) is a suitable approach for this threat. We may use the trust identity register to match the attributes of users' account values to deter this attack, as shown in Figure 9.

In our proposed system, to compute  $RT(CR, first(t), last(t))$ , which is the factor that represents the change rate of the trust result, we need to present a new procedure to

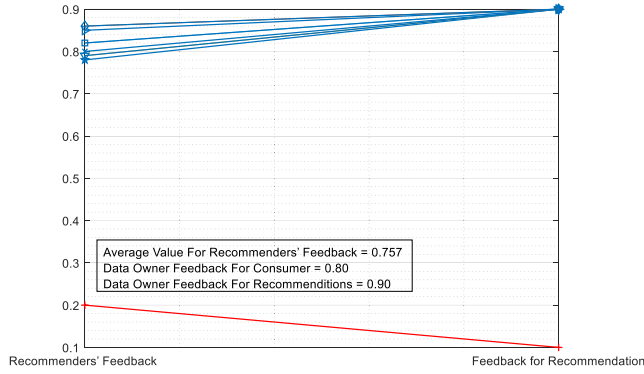


FIGURE 7. Feedback credibility.

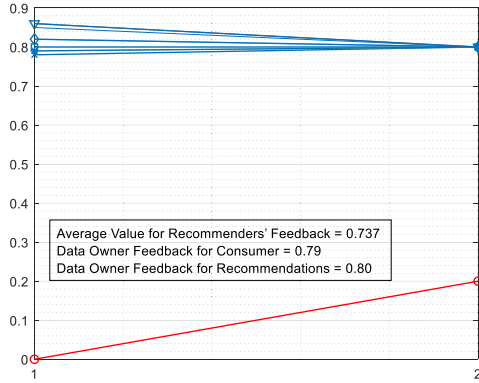


FIGURE 8. Procedure for recommendation aggregation.

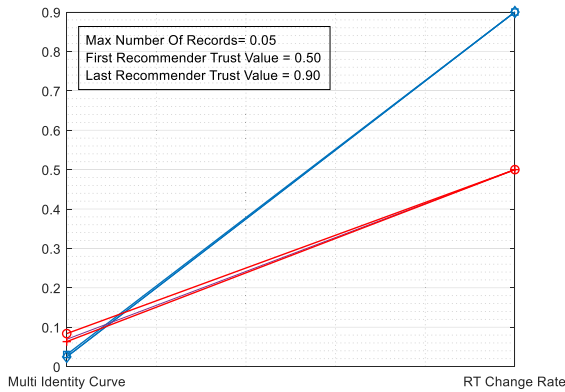


FIGURE 9. Multi-identity detection.

allow the TMS to ignore fake trust results for specific consumer roles ( $CR$ ). We must use the first recommendation trust value  $RT(first(t))$ , which represents the recommendation trust value for a specific consumer at a previous time. Second, we need to use the last recommendation trust value  $RT(last(t))$ , which represents the recommendation trust value for the same consumer at the most recent time the trust results were computed without considering the proposed method.

Figure 9 shows the impact of the first part of the multi-identity detection procedure, where in the first three cases, the trust model gives the same trust value for the

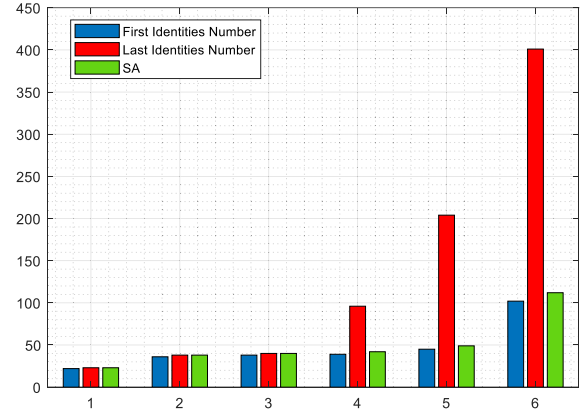


FIGURE 10. Multi-identity detection.

consumers in the last times because the identity number of these three cases is not greater than the maximum number of records that have the same credential attribute value. In the last three cases, the trust model gives the consumers the same trust value as in the first time because the number of identities of the three cases are greater than the maximum number of records that have the same credential attribute value.

To detect all these attacks, we must use the time factor, where the system will check for any changes in the behaviors of all recommenders in a short time  $RB(t)$ .  $|N_{id}(t)|$  represents the number of identities for all recommenders who have evaluated a specific consumer in a short time  $[first(t), last(t)]$ . We must measure the periodic and occasional percentages of all changes in the number of identities among the entire behavior of identities (the entire recognized identities for all recommenders who provided feedback for a specific consumer). Figure 10 shows the impact of the attack factor  $SA[first(t), last(t)]$  of a specified consumer in a short time.

Figure 10 shows the impact of the second part of the multi-identity detection, where the trust model will detect any change in the number of identities of all recommenders in a short time. In the first three cases, the trust model will not stop any identities whose number does not exceed the identity generation curve. In the last three cases, the trust model will stop all the identities that exceed the maximum identity generation curve.

In our system, we present recommender importance, defined as the particular importance of some SPs in their fields of interaction and their trustworthiness as recommenders. This weighs recommendations differently. Although most people care more about their own opinions, data owners might trust the recommendations of more experienced recommenders than themselves. As we can see in Figure 11, a weight function ( $W^{EX}$ ) must be added, multiplied by a number of exchange transactions ( $EX_i$ ) that a recommender has had with a specific consumer and dependent on the experience of recommender interactions  $W(R_i, CR)$ . Two additional factors should be considered. The time factor ( $t$ ) is a variable used to apply the impact of the

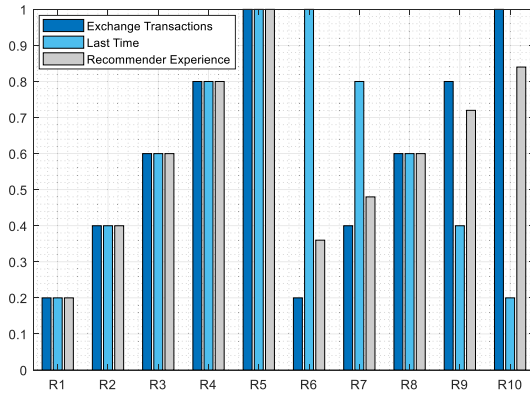


FIGURE 11. Recommender importance.

first time.  $Last(t)$  represents the last time that the recommender transacted with the intended consumer multiplied by the weight function ( $W^L$ ).

In Figure 11, we can notice how the time criterion and the number of transactions criterion affect the value of the recommender's expertise, where the trust model will compute the recommender importance for these 10 recommenders on the basis of their experience.

In a trust management system, a "subjectivity" problem occurs when recommenders have differing views and perceptions. Inaccurate user feedback is an example of a subjectivity problem. As users' differing views affect the accuracy of the interaction trust assessment, trust is a subjective term that may trigger serious problems. As a result, data owners must view the trust results of all consumers on the basis of their own viewpoints. As shown in Figure 12, a simple implementation of the computation aims to provide trust in the form of a percentile. A percentile calculation reflects a recommender's view of a specific user in conjunction with other trust values that have already been evaluated by the data owner. As a result, the data owner aims to change consumer trust and build interaction-related functional trust in a particular user, on the basis of their perception.  $I_R$  is a composite of the recommender's relationship confidence appraisal of all consumers' roles with which the recommender has transacted.

Consumer trust  $IT(R_i, CR)$  changes according to the  $SP$  perception  $S(R, CR)^{SP}$ . Figure 12 depicts the variations between the  $SP$  feedback  $I_{SP}$  and recommender feedback  $I_R$ , and the effects of these variables on  $IT(R_i, CR)$  and  $S(R, CR)^{SP}$ . Our model can eliminate subjectivity and provide correct results using these criteria.

To avoid collusion attacks, we used three criteria, each with several factors. Malicious recommendation detection ( $MRD$ ) is the first criterion that determines the likelihood that a suspected group is a collusion group. To detect these classes, we must quantify the time span of the feedback provided to detect all suspicious feedback, which is limited to anyone who sets out to attack a specific user. The TMS model then computes another criterion, malicious recommenders' behavior ( $MRB$ ), to measure the similarity of all recommenders' behaviors, which will be similar when

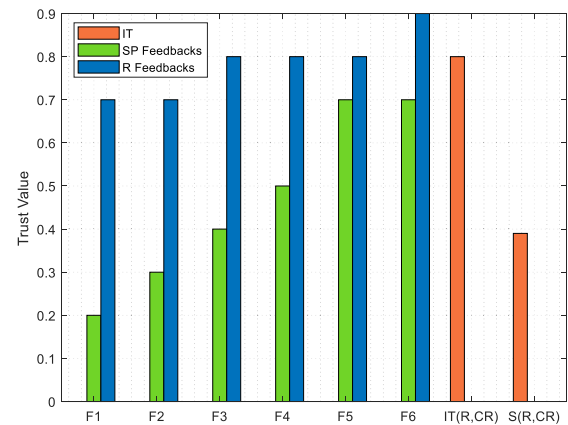


FIGURE 12. Subjectivity.

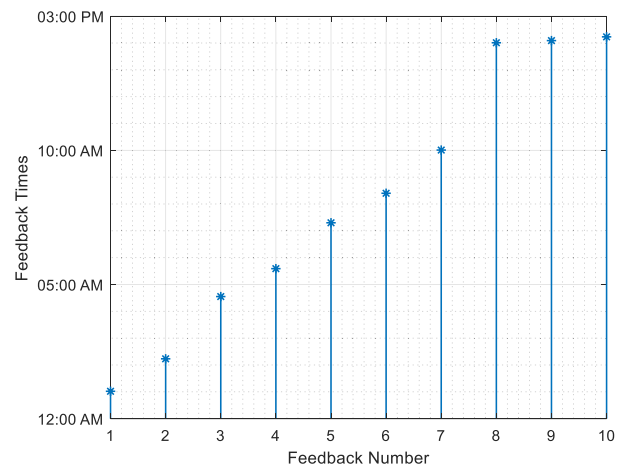


FIGURE 13. Malicious recommenders' behavior (MRB).

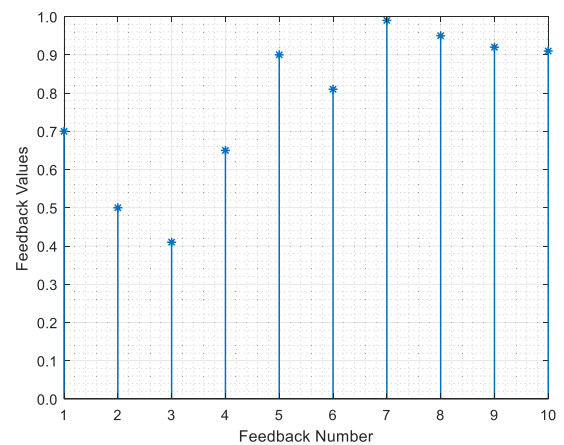


FIGURE 14. Malicious recommendation detection (MRD).

malicious recommenders target any user. The malicious recommendations detection ( $MRD$ ) and malicious recommenders' behavior ( $MRB$ ) are computed as follows:

In Figure 13 and 14, the trust model system will compare the time and value of all feedback items in the feedback set ( $FS$ ) that are given to a specific consumer. In the first

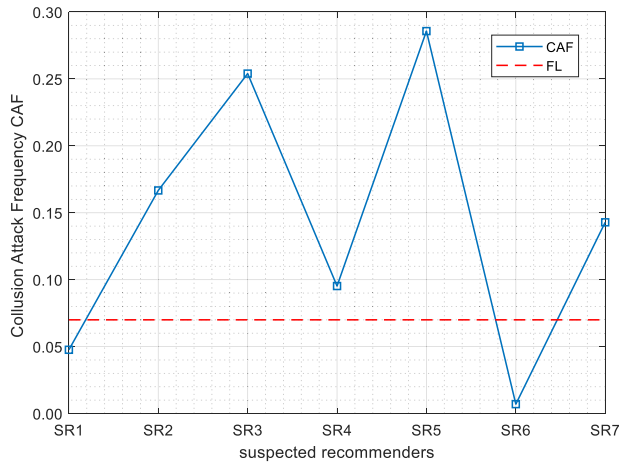


FIGURE 15. Collusion attack frequency.

comparison, the TMS will compare the time of the last feedback and the time of all feedback items except for the last feedback. After that, the TMS will compare the value of the last feedback and the value of all except the last feedback. Then, the trust model will compare the time and value of the last feedback in the feedback set ( $FS$ ) and the time and value of all feedback in the suspected set ( $SS$ ). In this way, the TMS will place all suspected feedback into the suspected set ( $SS$ ) on the basis of the range of feedback time curve  $TC$  and feedback value curve  $VC$ , respectively. In this case, the trust model will remove the suspected feedback (the last three feedback items in the feedback set in Figure 13 and 14) from the feedback set and transfer it to the suspected set. Then, the trust model will apply the third criterion, which is the collusion attack frequency ( $CAF$ ), to determine all collusion feedback by computing the  $CAF$  for all recommenders who have feedback items in the suspected feedback set to identify malicious recommendations, also called collusion feedback. Here, the strength of the attack is higher when the attack frequency is greater. The collusion attack frequency can be computed as shown in Figure 15.

In Figure 15, the trust model initially calculates the collusion attack frequency ( $CAF$ ), where  $FN(SR, CR)$  signifies the number of feedback items provided by a particular recommender to a particular consumer in the suspected set ( $SS$ ).  $FN(SS, CR)$  refers to the number of feedback items in the suspected set ( $SS$ ) that are provided to the same consumer. When the  $CAF$  is greater than the feedback limit ( $FL$ ), the trust model shifts the suspected feedback to the collusion set ( $CS$ ). However, if this is not the case, the suspected feedback  $SF(SR, CR)$  will be shifted by the trust model to the feedback set ( $FS$ ).

The trust model will examine various parameters that influence a consumer's trust value to recognize collusion attacks. The collusion attack frequency ( $CAF$ ) is one of these criteria in which the value of feedback frequency is directly related to the collusion feedback and inversely related to the aggregated feedback credibility. The value of feedback frequency

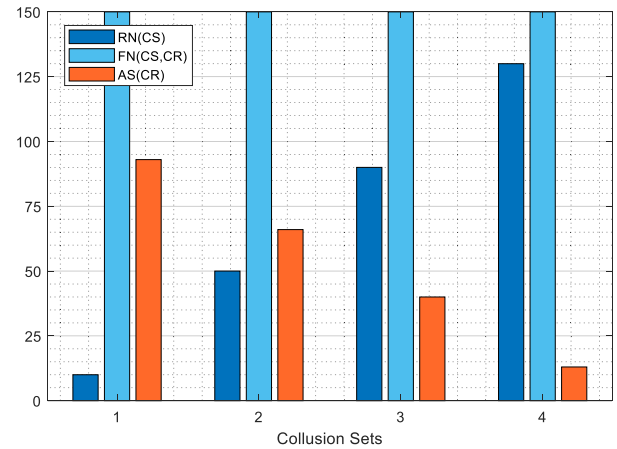


FIGURE 16. Attack scale (AS).

is determined by the number of feedback items given by each recommender and the total number of feedback items in the suspected set ( $SS$ ).

Suppose the frequency of seven suspected recommenders is as shown in Figure 15, and the feedback limit ( $FL$ ) is set to 7%. The feedback of any suspicious recommenders with a feedback frequency higher than that of the feedback limit ( $FL$ ) will be transferred to the collusion set ( $CS$ ); otherwise, the feedback will be relocated to the feedback set ( $FS$ ). Figure 15 shows five suspected recommenders who received feedback more frequently than the feedback limit ( $FL$ ).

In measuring the impact of the collusion attacks criterion, these results help the cloud administrator measure the scale of collusion attacks on cloud computing services.

In Figure 16, the trust model determines the size of the collusion set ( $CS$ ) to determine the attack scale ( $AS$ ) for a specific consumer, where the malicious recommenders from a recommender's community become a major percentage of all recommenders attacking the trust model and causing harm to it. To determine the collusion attack scale ( $AS$ ), which represents the size of the attack scale for different collusion sets, the trust model will divide the number of malicious recommenders within the collusion community by the number of malicious feedback items for all consumers within the collusion set ( $CS$ ) and subtract this number from 1.

Figure 17 shows the value of the attack target scale ( $ATS$ ), which provides the malicious feedback rate from one collusion set ( $CS$ ) for a specific consumer. To determine the attack target scale ( $ATS$ ), the trust model will divide the feedback number obtained from malicious recommenders by the feedback number that all feedback sets have been given, in which the set of malicious recommenders belongs to all recommenders who have evaluated a particular consumer.

Figure 18 shows the value of the collusion attack strength ( $CAS$ ) by calculating the rate of all malicious feedback from different collusion sets ( $CS$ ) for a specific consumer. For this, the value of all attack target scales ( $ATS$ ) is considered, after which the collusion attack strength ( $CAS$ ) is computed as shown in Figure 18, where the trust model will divide the



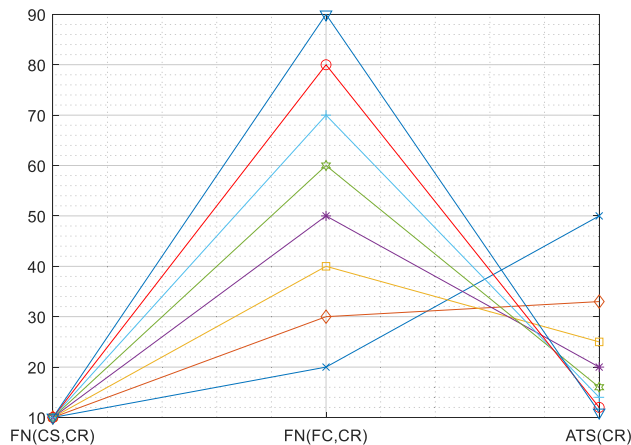


FIGURE 17. Attack target scale (ATS).

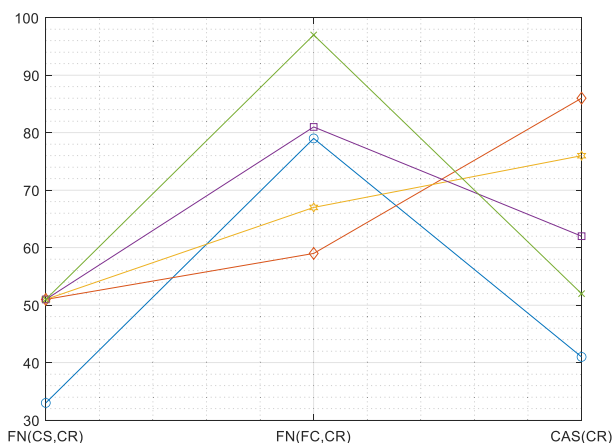


FIGURE 18. Collusion attack strength (CAS).

number of feedback items that has been obtained from malicious recommenders in all sets by the number of feedback items in all sets.

## VI. CONCLUSION

This study sought to identify a solution for dealing with trust issues in access control models to reduce risk. This could be done by making cloud storage systems more secure, and thus enhancing the quality of decisions taken by cloud operators and data owners. A terminology for trust criteria and reputation attacks in cloud computing was presented in this study. The study also puts forward a few of the key concepts related to trust management of services in cloud settings, as well as the most recently developed technologies. Each trust model has three layers, each consisting of a series of dimensions, which were further determined as the assessment criteria and benchmarks for the evaluation of many research prototypes of trust models in cloud computing environments.

Also, we demonstrated how to build a stable and versatile trust-based cloud storage system in which the owners of system resources may grant permission for roles and tasks, and the system will interrupt the task or role if there is a data leak. By implementing the T-RBAC as a new form of

access control combined with a robust trust model, we built a model that can reduce threats through high protection for cloud storage systems, which has improved the quality of decisions made by cloud operators and data owners.

## ACKNOWLEDGMENT

I, Salah T. Alshammari, would like to extend my heartfelt gratitude to my father for his unwavering support, guidance, and encouragement. His wisdom and belief in my abilities have been a constant source of strength throughout my academic and professional journey.

## REFERENCES

- [1] M. Ghafoorian, D. Abbasinezhad-Mood, and H. Shakeri, "A thorough trust and reputation based RBAC model for secure data storage in the cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 4, pp. 778–788, Apr. 2019, doi: 10.1109/TPDS.2018.2870652.
- [2] T. H. Noor, Q. Z. Sheng, and A. Alfazi, "Reputation attacks detection for effective trust assessment among cloud services," in *Proc. 12th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, Jul. 2013, pp. 469–476.
- [3] T. H. Noor, Q. Z. Sheng, and A. Bouguettaya, *Trust Management in Cloud Services*. Cham, Switzerland: Springer, 2014, pp. 1–119.
- [4] O. Hasan, L. Brunie, and E. Bertino, "Preserving privacy of feedback providers in decentralized reputation systems," *Comput. Secur.*, vol. 31, no. 7, pp. 816–826, Oct. 2012.
- [5] L. Zhou, V. Varadarajan, and M. Hitchens, "Integrating trust with cryptographic role-based access control for secure cloud data storage," in *Proc. 12th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, Jul. 2013, pp. 560–569.
- [6] V. Mali and P. Patil, "A survey on authentication and access control for cloud computing using RBAC mechanism," *Int. J. Innov. Res. Comput. Commun. Eng.*, vol. 12, no. 3, pp. 12125–12129, 2015.
- [7] P. Zhang, Y. Kong, and M. Zhou, "A domain partition-based trust model for unreliable clouds," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 9, pp. 2167–2178, Sep. 2018.
- [8] N. Iltaf, A. Ghafoor, and M. Hussain, "Modeling interaction using trust and recommendation in ubiquitous computing environment," *EURASIP J. Wireless Commun. Netw.*, vol. 2012, no. 1, pp. 1–13, Dec. 2012.
- [9] Z. Tan, Z. Tang, R. Li, A. Sallam, and L. Yang, "Research on trust-based access control model in cloud computing," in *Proc. 6th IEEE Joint Int. Inf. Technol. Artif. Intell. Conf.*, vol. 2, Aug. 2011, pp. 339–344.
- [10] A. Barsoum and A. Hasan, "Enabling dynamic data and indirect mutual trust for cloud computing storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 12, pp. 2375–2385, Dec. 2013.
- [11] W. W. Smari, P. Clemente, and J.-F. Lalande, "An extended attribute based access control model with trust and privacy: Application to a collaborative crisis management system," *Future Gener. Comput. Syst.*, vol. 31, pp. 147–168, Feb. 2014.
- [12] E. W. Michael and H. J. Mattord, "Implementing information security," *CENGAGE Learn.*, pp. 433–469, 2012.
- [13] M. Uddin, S. Islam, and A. Al-Nemrat, "A dynamic access control model using authorising workflow and task-role-based access control," *IEEE Access*, vol. 7, pp. 166676–166689, 2019.
- [14] X. Li and J. Du, "Adaptive and attribute-based trust model for service-level agreement guarantee in cloud computing," *IET Inf. Secur.*, vol. 7, no. 1, pp. 39–50, Mar. 2013.
- [15] H. Yu, Z. Shen, C. Miao, C. Leung, and D. Niyato, "A survey of trust and reputation management systems in wireless communications," *Proc. IEEE*, vol. 98, no. 10, pp. 1755–1772, Oct. 2010.
- [16] W. Chang, F. Xu, and J. Dou, "A trust and unauthorized operation based RBAC (TUORBAC) model," in *Proc. Int. Conf. Control Eng. Commun. Technol.*, Dec. 2012, pp. 811–814.
- [17] S. T. Alshammari, A. Albeshri, and K. Alsubhi, "Integrating a high-reliability multicriteria trust evaluation model with task role-based access control for cloud services," *Symmetry*, vol. 13, no. 3, p. 492, Mar. 2021.
- [18] S. T. Alshammari, A. Albeshri, and K. Alsubhi, "Building a trust model system to avoid cloud services reputation attacks," *Egyptian Informat. J.*, vol. 22, no. 4, pp. 493–503, Dec. 2021.

- [19] S. T. Alshammari, K. Alsubhi, H. M. A. Aljahdali, and A. M. Alghamdi, "Trust management systems in cloud services environment: Taxonomy of reputation attacks and defense mechanisms," *IEEE Access*, vol. 9, pp. 161488–161506, 2021.
- [20] S. T. Alshammari and K. Alsubhi, "Building a reputation attack detector for effective trust evaluation in a cloud services environment," *Appl. Sci.*, vol. 11, no. 18, p. 8496, Sep. 2021.
- [21] A. Ali, *Context-aware Access Control in Ubiquitous Computing (CRAAC)*, 2010.
- [22] M. Alam, N. Emmanuel, T. Khan, Y. Xiang, and H. Hassan, "Garbled role-based access control in the cloud," *J. Ambient Intell. Humanized Comput.*, vol. 9, no. 4, pp. 1153–1166, Aug. 2018.
- [23] L. Huang, Z. Xiong, and G. Wang, "A trust-role access control model facing cloud computing," in *Proc. 35th Chin. Control Conf. (CCC)*, Jul. 2016, pp. 5239–5242.
- [24] S. Chakraborty and I. Ray, "TrustBAC: Integrating trust relationships into the RBAC model for access control in open systems," in *Proc. 11th ACM Symp. Access Control Models Technol.*, 2006, pp. 49–58.
- [25] W. Deng and Z. Zhou, "A flexible RBAC model based on trust in open system," in *Proc. 3rd Global Congr. Intell. Syst.*, Nov. 2012, pp. 400–404.
- [26] P. Wang and L. Jiang, "Task-role-based access control model in smart health-care system," in *Proc. MATEC Web Conf.*, vol. 22, 2015, p. 1011.
- [27] S. Oh and S. Park, "Task-role-based access control model," *Inf. Syst.*, vol. 6, no. 6, pp. 533–562, 2003.
- [28] A. Gholami, "Security and privacy of sensitive data in cloud computing," Ph.D. dissertation, KTH Roy. Inst. Technol., Stockholm, Sweden, 2016.
- [29] P. Gupta, "Trust and reliability management in large-scale cloud computing environments," in *Large-Scale Data Streaming, Processing, and Blockchain Security*. Hershey, PA, USA: IGI Global, 2020, pp. 66–89.
- [30] L. Zhou, V. Varadharajan, and M. Hitchens, "Trust enhanced cryptographic role-based access control for secure cloud data storage," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 11, pp. 2381–2395, Nov. 2015.
- [31] T. H. Noor, Q. Z. Sheng, L. Yao, S. Dustdar, and A. H. H. Ngu, "CloudArmor: Supporting reputation-based trust management for cloud services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 2, pp. 367–380, Feb. 2016.
- [32] M. Alhamad, T. Dillon, and E. Chang, "SLA-based trust model for cloud computing," in *Proc. 13th Int. Conf. Netw.-Based Inf. Syst.*, Sep. 2010, pp. 321–324.
- [33] N. Santos, K. P. Gummadi, and R. Rodrigues, "Towards trusted cloud computing," *HotCloud*, vol. 9, no. 9, p. 3, 2009.
- [34] K. Hwang, S. Kulkareni, and Y. Hu, "Cloud security with virtualized defense and reputation-based trust mangement," in *Proc. 8th IEEE Int. Conf. Dependable, Autonomic Secure Comput.*, Dec. 2009, pp. 717–722.
- [35] K. Hwang and D. Li, "Trusted cloud computing with secure resources and data coloring," *IEEE Internet Comput.*, vol. 14, no. 5, pp. 14–22, Sep. 2010.
- [36] C. Uikey and D. S. Bhilare, "TrustRBAC: Trust role based access control model in multi-domain cloud environments," in *Proc. Int. Conf. Inf. Commun., Instrum. Control (ICICIC)*, Aug. 2017, pp. 1–7.
- [37] E. Koutrouli and A. Tsalgatidou, "Taxonomy of attacks and defense mechanisms in P2P reputation systems—Lessons for reputation system designers," *Comput. Sci. Rev.*, vol. 6, nos. 2–3, pp. 47–70, May 2012.
- [38] A. Ugur, "Manipulator: A novel collusion attack on trust management systems in social IoT," in *Proc. Comput. Sci. On-line Conf.*, Cham, Switzerland: Springer, 2021, pp. 578–592.
- [39] O. Hasan, L. Brunie, J.-M. Pierson, and E. Bertino, "Elimination of sub-jectivity from trust recommendation," *Int. Fed. Inf. Process.*, pp. 65–80, Jun. 2009.



research interests include information security, artificial intelligence, cybersecurity, data security in cloud computing, data analytics, trust in cloud computing, and software testing.

**MUNA AL-RAZGAN** received the Ph.D. degree in information technology from George Mason University, Fairfax, VA, USA. She is currently working as an Associate Professor of software engineering with the College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. Her research interests include data mining, machine learning, artificial intelligence, educational data mining, and assistive technologies.



**TAHA ALFAKIH** received the B.S. degree in computer science from the Department of Computer Science, Hadhramout University, Yemen, the M.Sc. degree from the Department of Computer Science, King Saud University (KSU), Riyadh, Saudi Arabia, and the Ph.D. degree from the Department of Information System, KSU. He also works as a Researcher with the College of Computer Science, KSU. His research interests include machine learning, mobile edge computing, and the Internet of Things (IoT).



**KHALID A. ALGHAMDI** received the M.Sc. degree in computer science from Al-Imam Mohammad Ibn Saud Islamic University, Alriyadh, Saudi Arabia, in 2016. Currently, he serves as a Senior Consultant at the Public Technology Governance Authority, Riyadh, Saudi Arabia. His research interests span inclusive cloud-based big data applications, cybersecurity data mining, machine learning, and artificial intelligence.

...